

**EVALUASI MULTI-DIMENSI UNTUK POLA ARSITEKTUR
MODERN BACKEND PADA APLIKASI DATA-INTENSIF**
(Studi Kasus: Vertical Slice Implementation pada Backend Aplikasi *Point of Sale*)

PROPOSAL SKRIPSI

Sebagai Salah Satu Syarat Untuk Memperoleh Gelar Sarjana Komputer



Disusun Oleh :
MUHAMAD OSKHAR MUBAROK
11220910000042

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UIN SYARIF HIDAYATULLAH JAKARTA**

LEMBAR PERSETUJUAN

EVALUASI MULTI-DIMENSI UNTUK POLA ARSITEKTUR MODERN BACKEND PADA APLIKASI DATA-INTENSIF

(Studi Kasus: Vertical Slice Implementation pada Backend Aplikasi *Point of Sale*)

Proposal Tugas Akhir

Sebagai Salah Satu Syarat Untuk Memperoleh Gelar Sarjana Komputer

Oleh :

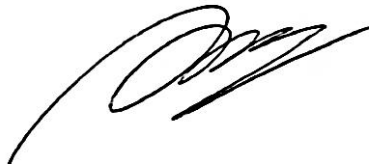
MUHAMAD OSKHAR MUBAROK

11220910000042

Menyetujui:

Calon Dosen Pembimbing I

Calon Dosen Pembimbing II



Dr. Imam Marzuki Shofi M.T.

NIP. 197202052008011010

Belum Ada

KATA PENGANTAR

Puji syukur penyusun panjatkan kepada Allah Subhanahu wa Ta'ala Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penyusun dapat menyelesaikan proposal skripsi ini yang berjudul "Evaluasi Multi-Dimensi Arsitektur Pattern Modern Backend pada Aplikasi Data-Intensif" tepat pada waktunya tanpa hambatan yang berarti dan sesuai dengan yang diharapkan. Terima kasih yang sebesar-besarnya penyusun sampaikan kepada Dr. Imam Marzuki Shofi M.T., selaku calon pembimbing skripsi yang telah memberikan bimbingan serta wawasan yang berharga dalam penyusunan proposal ini. Arahan dan dukungan beliau sangat membantu penyusun dalam memahami cara penulisan yang baik, konsep arsitektur sistem backend yang scalable dan berkinerja tinggi, serta penerapan design pattern, sehingga proposal ini dapat disusun dengan baik.

Penyusun menyadari bahwa proposal ini masih memiliki banyak kekurangan karena keterbatasan penyusun dalam waktu dan pengetahuan. Oleh karena itu, penyusun sangat mengharapkan kritik dan saran yang membangun untuk perbaikan proposal ini di masa mendatang. Semoga proposal ini dapat memberikan manfaat bagi semua pihak yang memerlukannya dan menjadi referensi yang berguna dalam pengembangan ilmu pengetahuan di bidang teknologi informasi dan pengembangan sistem backend modern.

Akhir kata, penyusun berharap proposal ini dapat menjadi sumbangsih yang berarti bagi pengembangan arsitektur sistem backend yang scalable dan berkinerja tinggi serta memberikan kontribusi positif dalam pengelolaan aplikasi data-intensif melalui penerapan design pattern yang tepat.

Jakarta, 10 Agustus 2025



Muhamad Oskhar Mubarak

NIM. 11220910000042

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI.....	iii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	5
1.6 Metodologi Penelitian	5
1.7 Tahap Implementasi	6
1.8 Sistematika Penulisan	7
BAB II.....	8
LANDASAN TEORI.....	8
2.1 Model C4 untuk Visualisasi Arsitektur.....	8
2.2 Arsitektur microservices	11
2.3 Domain-Driven Design (DDD).....	15
2.4 Manajemen Data pada Sistem Data-Intensif.....	18
2.5 Event Sourcing.....	21
2.6 Command Query Responsibility Segregation (CQRS).....	23
2.7 Perbandingan Microservices dan Monolith	26
2.8 Pengujian Dimensi Teknis & Kinerja Aplikasi.....	28
2.9 Rapid Application Development (RAD).....	32
2.10 Pengujian Dimensi Arsitektural	34

2.11	Pengujian Dimensi Developer	36
2.12	Studi Sejenis.....	40
BAB III		45
METODOLOGI PENELITIAN.....		45
3.1	Objek Penelitian	45
3.2	Tahap Implementasi	46
3.2.1	Spesifikasi Kebutuhan (Vertical Slice)	46
3.2.2	Standarisasi Repositori (Source Code Standardization - SCS)...	46
3.2.3	Implementasi Kode (Monolith ke Hybrid Vertical Slice).....	46
3.2.4	Penerapan CQRS dan Event Sourcing	49
3.2.5	Instrumentasi dan Pencatatan Metrik Developer	49
3.2.6	Unit Testing dan Integration Testing	50
3.2.7	Setup Pengujian Beban dan Observabilitas	50
3.2.8	Continuous Development Cycle	50
3.2.9	Deliverable dan Artefak Penelitian	50
3.3	Tahap Penelitian.....	51
3.3.1	Perancangan eksperimen & definisi variable	51
3.3.2	Persiapan lingkungan eksperimen & artefak reproduktif	51
3.3.3	Pembuatan dataset sintetis & profil beban	51
3.3.4	Implementasi vertical slice Monolith lalu Hybrid	51
3.3.5	Instrumentasi observabilitas & pencatatan metrik	52
3.3.6	Pengujian teknis (load testing) dengan prosedur rinci	52
3.3.7	Pengujian dimensi arsitektural	52
3.3.8	Failure injection & uji ketahanan operasional	53
3.3.9	Pengukuran metrik developer (praktik & ekstraksi)	53
3.3.10	Pengumpulan, validasi & penjagaan kualitas data.....	53
3.3.11	Analisis data & uji statistic	53
3.3.12	Dokumentasi, artefak akhir & reproducibility penelitian	54

3.4	Kerangka Penelitian	54
DAFTAR PUSTAKA		55

BAB I

PENDAHULUAN

1.1 Latar Belakang

Cakupan rekayasa perangkat lunak kontemporer tidak lagi dibatasi oleh kapasitas komputasi semata, melainkan oleh tantangan yang melekat pada data itu sendiri. Kita telah memasuki era *data-intensive application*, yakni sistem yang kendala utamanya terletak pada volume, kompleksitas, dan kecepatan perubahan data yang terus meningkat. Tantangan ini melampaui sekadar siklus CPU dan menyentuh fondasi bagaimana sistem dirancang agar dapat beroperasi secara andal dalam jangka panjang. Dalam kerangka ini, terdapat tiga pilar utama yang menjadi tolok ukur kualitas sistem modern: Reliabilitas (kemampuan untuk terus berfungsi dengan benar meskipun terjadi kesalahan), Skalabilitas (kemampuan untuk menangani peningkatan beban secara wajar), dan Maintainabilitas (kemampuan untuk dioperasikan, dipahami, dan dikembangkan dengan mudah oleh tim rekayasa). Kegagalan memenuhi salah satu dari atribut kualitas ini dapat berujung pada degradasi layanan, kegagalan sistem, bahkan kegagalan bisnis (Kleppmann, 2017).

Menanggapi tantangan fundamental tersebut, literatur mutakhir mengajukan solusi arsitektural yang melampaui pendekatan parsial. Salah satu usulan yang banyak diadopsi adalah mengintegrasikan *design pattern* modern seperti *Microservices*, *Command Query Responsibility Segregation* (CQRS), dan *Event Sourcing* (ES) ke dalam satu kerangka arsitektur hibrida. Secara teoretis, sinergi ketiga pola ini diyakini mampu menghasilkan sistem yang tangguh, berkinerja tinggi, dapat diaudit, dan mudah dikembangkan di masa depan. Misalnya, *Microservices* memungkinkan dekomposisi aplikasi menjadi serangkaian layanan independen yang masing-masing melayani domain bisnis tertentu (Lewis, James; Fowler, 2014), sehingga memudahkan pengembangan, *deployment*, dan *scaling* terpisah. Pendekatan ini efektif dalam menangani sistem *big data* karena dapat menskalakan hanya komponen yang dibutuhkan. Dari sisi *maintainability*, dekomposisi ini mengurangi beban kognitif tim pengembang, memungkinkan mereka fokus pada basis kode yang lebih kecil dan terfokus (Ataei & Staegemann, 2023).

Di sisi lain, CQRS hadir sebagai solusi terhadap perbedaan kebutuhan jalur *read* dan *write* dalam satu model data. Pemisahan tanggung jawab ini memungkinkan jalur *command* dioptimalkan untuk validasi, konsistensi, dan eksekusi perubahan data,

sementara jalur *query* dapat dirancang menggunakan model terdenormalisasi yang dioptimalkan untuk kecepatan pembacaan. Hal ini penting dalam aplikasi *data-intensive*, di mana beban baca sering kali jauh melampaui beban tulis (Kabbedijk et al., 2012). Kombinasi *Microservices*, CQRS, dan Event Sourcing dapat menangani beban tinggi secara efektif (Fansha et al., 2021). Sebagai pilar terakhir, Event Sourcing menawarkan paradigma penyimpanan data berbasis rangkaian peristiwa (*event stream*) yang bersifat *immutable* (Fowler, 2005). Setiap perubahan disimpan sebagai event yang dapat diputar ulang (*replay*) untuk merekonstruksi keadaan sistem pada titik waktu tertentu, sehingga mendukung *recovery* yang presisi dan kemampuan audit yang tinggi (N'guessan et al., 2023).

Namun, meskipun literatur dan praktik industri sering memposisikan kombinasi pola arsitektur ini sebagai “resep unggul” untuk aplikasi data-intensive, asumsi tersebut berpotensi bias. Terdapat kecenderungan untuk menggeneralisasi bahwa menggabungkan beberapa konsep “baik” akan selalu menghasilkan hasil “superior”. Dalam kenyataan, setiap keputusan arsitektural membawa trade-off yang tidak dapat diabaikan. Efisiensi di satu aspek sering kali dibayar dengan kompleksitas atau biaya di aspek lain. Misalnya, *Microservices* memang meningkatkan skalabilitas, tetapi menambah kompleksitas orkestrasi; CQRS mengoptimalkan kinerja baca/tulis tetapi meningkatkan kebutuhan sinkronisasi antar-model; Event Sourcing memberikan reliabilitas dan auditabilitas, namun dapat menimbulkan storage footprint besar serta waktu rehidrasi (state rehydration time) yang signifikan (Overeem et al., 2021).

Konteks penelitian ini difokuskan pada aplikasi Point of Sale (POS), yang secara alami bersifat *data-intensive*. Sistem POS harus menangani volume transaksi tinggi, kebutuhan pembacaan data cepat (misalnya untuk pencarian produk dan laporan penjualan), serta integrasi dengan berbagai modul seperti inventori, manajemen pelanggan, dan sistem pembayaran. Karakteristik ini membuat POS menjadi studi kasus ideal untuk menguji efektivitas arsitektur backend modern dalam lingkungan yang mensimulasikan skenario industri nyata.

Untuk mendapatkan gambaran yang obyektif, dibutuhkan baseline perbandingan yang jelas. Dalam konteks ini, arsitektur monolith menjadi tolok ukur (*baseline*) yang relevan karena sifatnya yang lebih sederhana, terintegrasi, dan telah lama digunakan secara luas di industri. Dengan membandingkan implementasi kombinasi *Microservices* + CQRS + *Event Sourcing* terhadap monolith, penelitian dapat mengungkap perbedaan nyata dalam hal kinerja, karakteristik arsitektural, dan beban kerja pengembang.

Oleh karena itu, penelitian ini memposisikan diri bukan untuk sekadar mengonfirmasi superioritas teori, melainkan untuk melakukan evaluasi multi-dimensi yang komprehensif terhadap pola arsitektur *Microservices*, *CQRS*, dan *Event Sourcing* pada konteks backend aplikasi *data-intensive*. Evaluasi ini mencakup tiga dimensi utama. Pertama, dimensi teknis dan kinerja aplikasi, dengan metrik seperti *throughput*, latensi (p50, p95, p99), *success rate* (termasuk tingkat *error recovery*), serta penggunaan CPU dan memori. Kedua, dimensi arsitektural, yang berfokus pada karakteristik khas seperti *eventual consistency lag*, *state rehydration time*, dan *storage footprint* untuk mengukur kematangan pemahaman arsitektur. Ketiga, Dimensi Developer, yang secara objektif mengukur pengalaman pengembangan melalui metrik seperti *lead time for changes* (mengacu pada standar SCS), *commit granularity & churn*, serta *context-switching count* sebagai indikator beban kognitif dan potensi “neraka koreografi” (*choreography hell*) (Taibi & Systä, 2020). Pendekatan ini diharapkan menghasilkan gambaran yang utuh tentang manfaat dan biaya (*cost-benefit*) implementasi pola arsitektur modern pada aplikasi *data-intensive*, sehingga dapat menjadi panduan berbasis data bagi pengambil keputusan di industri.

1.2 Rumusan Masalah

Berdasarkan uraian pada latar belakang, penelitian ini dilandasi oleh kritik terhadap asumsi umum di dunia industri bahwa penggabungan beberapa pola arsitektur yang dianggap “baik” akan secara otomatis menghasilkan sistem yang “superior”. Kenyataannya, setiap keputusan arsitektural memiliki konsekuensi dan *trade-off* yang perlu dianalisis secara holistik. Permasalahan yang ingin dipecahkan melalui penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana mengukur performa teknis pola arsitektur *Microservices*, *CQRS*, dan *Event Sourcing* pada aplikasi *data-intensive* secara objektif?
2. Bagaimana mengidentifikasi *trade-off* arsitektural yang muncul, meliputi karakteristik seperti *eventual consistency lag*, *state rehydration time*, dan *storage footprint*?
3. Bagaimana mengukur dampak pola arsitektur tersebut terhadap pengalaman pengembang (*developer experience*), termasuk beban kognitif, granularitas komit, dan waktu penyelesaian perubahan (*lead time for changes*)?

1.3 Batasan Masalah

Penelitian ini membatasi ruang lingkup pada hal-hal berikut:

1. Lingkup Sistem yang Dievaluasi

Evaluasi dilakukan pada dua pendekatan arsitektur backend aplikasi Point of Sale dengan karakteristik data-intensive, yaitu:

- a. Kombinasi Microservices + CQRS + Event Sourcing
- b. Monolith sebagai baseline pembanding

2. Lingkup Teknis

Implementasi dilakukan dalam bentuk *Vertical Slice* yang representatif untuk kedua arsitektur. Pengujian dilakukan di lingkungan *staging* terisolasi, bukan di sistem produksi.

3. Lingkup Evaluasi

- a. Dimensi Teknis & Kinerja Aplikasi → throughput, latency (p50, p95, p99), success rate, penggunaan CPU & memori.
- b. Dimensi Arsitektural → eventual consistency lag, state rehydration time, storage footprint.
- c. Dimensi Developer → lead time for changes, commit granularity & churn, context-switching count.

4. Lingkup Tools & Teknologi

Pengembangan backend menggunakan Node.js dan NestJS, dengan desain arsitektur divisualisasikan menggunakan C4 Model. Pengujian kinerja dilakukan menggunakan Artillery sebagai *load testing tool*.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengevaluasi secara multi-dimensi performa pola arsitektur modern backend (*Microservices*, *CQRS*, *Event Sourcing*) pada aplikasi *data-intensive*.
2. Mengidentifikasi *keuntungan* dan konsekuensi implementasi pola arsitektur tersebut dalam konteks backend aplikasi Point of Sale.
3. Memberikan panduan berbasis data yang dapat membantu pengambil keputusan arsitektural di industri untuk memilih pola yang sesuai berdasarkan kebutuhan dan konteks.

1.5 Manfaat Penelitian

Penelitian ini diharapkan menghasilkan:

1. Bagi Penulis
 - a. Mendapatkan gelar Sarjana Komputer Strata 1
 - b. Menambah wawasan dan pemahaman yang lebih mendalam terkait topik penelitian.
 - c. Melatih kemampuan dalam menganalisis, merancang, serta mengimplementasikan solusi sesuai metode penelitian.
2. Bagi Pembaca
 - a. Memberikan referensi tambahan terkait topik yang dibahas sehingga dapat memperluas pengetahuan pembaca.
 - b. Menjadi bahan acuan untuk penelitian selanjutnya dengan tema yang sejenis.
 - c. Menyediakan gambaran praktis mengenai penerapan teori ke dalam implementasi nyata.
3. Bagi Universitas
 - a. Menambah koleksi karya ilmiah yang dapat digunakan sebagai referensi akademik.
 - b. Meningkatkan kontribusi universitas dalam menghasilkan penelitian yang bermanfaat.
 - c. Menjadi salah satu bukti implementasi kurikulum dalam menghasilkan penelitian yang relevan dengan perkembangan ilmu pengetahuan.
4. Bagi Industri
 - a. Memberikan alternatif solusi atau inspirasi yang dapat diterapkan dalam praktik industri.
 - b. Menjadi bahan pertimbangan dalam pengembangan pemilihan arsitektural yang sejalan dengan kebutuhan industri.
 - c. Mendapatkan referensi bagaimana melakukan evaluasi pada pola arsitektur software yang ingin diimplementasikan di suatu industri.

1.6 Metodologi Penelitian

Untuk mencapai tujuan tersebut, penelitian ini menggunakan metodologi eksperimen terkontrol yang meliputi:

1. Eksperimen Terukur
 - a. Arsitektur sudah ter-implementasi
 - b. Melakukan uji beban (load testing) untuk mengukur throughput, latency (p50, p95, p99), dan success rate.
 - c. Melakukan pengujian konsistensi, rehidrasi state, serta footprint penyimpanan untuk mengidentifikasi trade-off arsitektural.
2. Pengumpulan Data
 - a. Mencatat hasil pengujian performa dan arsitektur.
 - b. Mengumpulkan metrik developer dari standar SCS (lead time for changes, commit granularity, churn, context-switching).
3. Evaluasi dan Hasil
 - a. Menganalisis data dari ketiga dimensi: teknis & performa, arsitektural, dan pengembang.
 - b. Menyusun hasil perbandingan terhadap baseline monolith untuk menilai manfaat dan biaya dari penggabungan arsitektur modern.

1.7 Tahap Implementasi

Sebelum dilakukan penelitian utama, sistem terlebih dahulu diimplementasikan sebagai objek uji. Implementasi ini mengikuti pendekatan Rapid Application Development (RAD) dan secara paralel mencatat metrik pengembang. Tahapan inti dari implementasi dapat diringkas sebagai berikut:

1. Requirements Specification untuk Mendefinisikan kebutuhan sistem Vertical Slice dari Point of Sale (POS) sebagai studi kasus.
2. Source Code Standardization (SCS) via GitHub untuk Menetapkan standar repositori untuk pencatatan commit dan versioning.
3. Code Implementation & Version Control untuk Mengembangkan vertical slice arsitektur monolith dan hybrid (Microservices + CQRS + ES).
4. Developer Metrics Logging untuk Mencatat metrik pengembang: lead time, commit granularity & churn, context switching.
5. Unit & Integration Testing untuk Memastikan fungsionalitas dasar sistem berjalan sebelum pengujian penelitian.
6. Continuous Development Cycle untuk Menyempurnakan implementasi sesuai kebutuhan, sampai siap diuji dalam metodologi penelitian.

1.8 Sistematika Penulisan

Penulis menyusun penelitian ini dengan sistematika penulisan yang akan melakukan pembahasan menjadi 6 bab, yaitu :

BAB I PENDAHULUAN

Bab ini berisi latar belakang masalah, perumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, metodologi penelitian, dan sistematika penulisan.

BAB II LANDASAN TEORI

Bab ini berisi teori-teori yang berhubungan dengan konsep penulisan seperti teori dasar, dan beberapa konsep penting mengenai topik.

BAB III METODOLOGI PENELITIAN

Bab ini berisi langkah-langkah metode yang akan digunakan dalam metodologi penelitian.

BAB IV IMPLEMENTASI DAN TESTING

Pada bab ini menjelaskan hasil implementasi dan menjelaskan rincian evaluasi serta testing yang dilakukan

BAB V HASIL DAN PEMBAHASAN

Bab ini berisi mengenai hasil akhir dari seluruh evaluasi dan investigasi yang dilakukan terhadap pola arsitektur yang diuji.

BAB VI PENUTUP

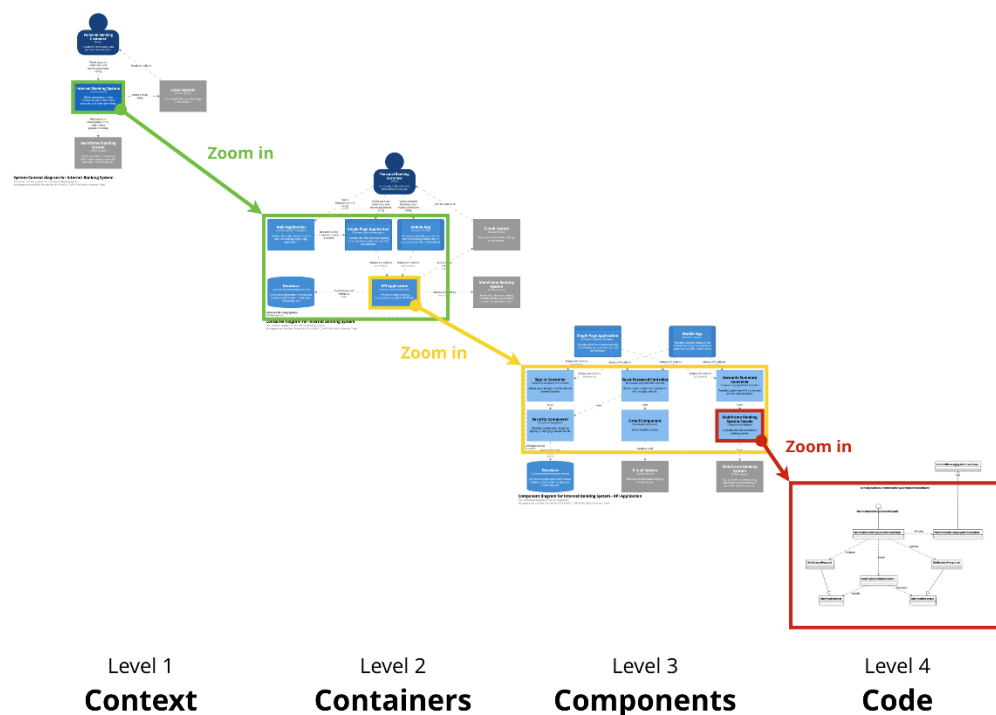
Bab ini berisi kesimpulan dan saran dari penelitian yang telah dilakukan yang dapat diperhatikan untuk penelitian selanjutnya yang lebih baik lagi.

BAB II

LANDASAN TEORI

2.1 Model C4 untuk Visualisasi Arsitektur

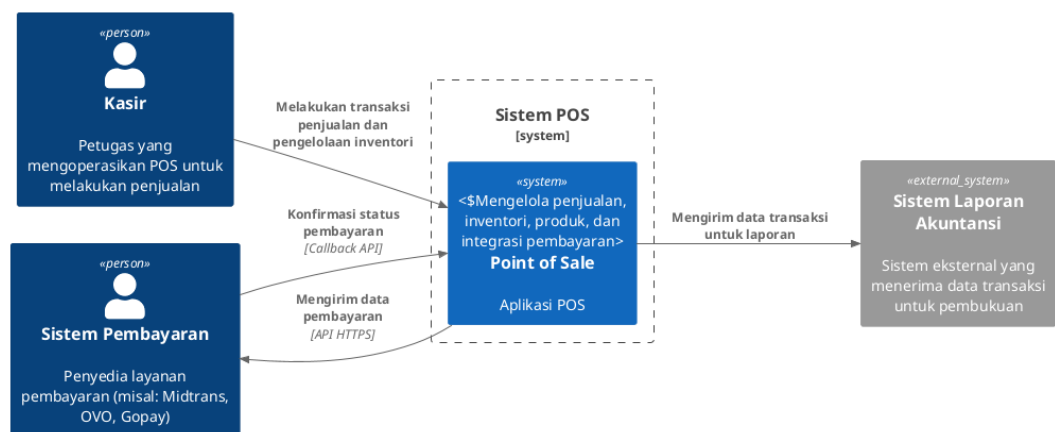
Model C4 menawarkan cara yang terstruktur dan bertingkat untuk memvisualisasikan arsitektur perangkat lunak sehingga berbagai pemangku kepentingan dapat memahami sistem pada tingkat detail yang sesuai dengan peran mereka. Pada level paling atas model C4 menyajikan diagram konteks yang memetakan sistem terhadap aktor eksternal, pada level berikutnya diagram container menggambarkan bagian-bagian yang menjalankan proses (misalnya layanan, database, dan komponen eksternal), lalu diagram komponen memecah container menjadi unit-unit fungsional, dan akhirnya diagram kode yang menampilkan rincian implementasi. (Vazquez-Ingelmo et al., 2020) menegaskan bahwa struktur berlapis ini memudahkan pembelajaran arsitektur oleh mahasiswa dan praktisi karena memungkinkan proses pembelajaran yang bertahap dari gambaran umum menuju detail implementasi.



Gambar 2.1. Keseluruhan bentuk dari diagram C4 Model

Penggunaan model C4 pada konteks microservices dan domain-driven design memungkinkan pemetaan bounded context langsung ke container dan komponen yang terdefinisi, sehingga hubungan konseptual domain dapat ditransformasikan menjadi artefak arsitektural yang dapat dieksekusi. (Levezinho et al., 2024) menunjukkan bahwa ketika melakukan dekomposisi monolith menjadi layanan mikro, analisis pola akses entitas dapat diterjemahkan menjadi kandidat bounded context yang kemudian dapat dipresentasikan secara eksplisit di diagram C4. Untuk penelitian POS ini, langkah formal tersebut berarti setiap bounded context yang diidentifikasi melalui analisis akses data dapat direpresentasikan sebagai container dalam diagram C4, memudahkan perbandingan langsung antara implementasi monolith dan microservices pada level struktur sistem.

Model C4 juga unggul dalam mendukung evolusi arsitektur dan rekayasa ulang karena diagramnya fokus pada elemen-elemen yang relevan untuk keputusan desain praktis. (Keller et al., 1999) dalam kajiannya tentang reverse-engineering menekankan pentingnya representasi arsitektural yang memfasilitasi identifikasi komponen dan dependensi saat melakukan refaktorisasi. Karena C4 menampilkan container dan komponen yang berorientasi pada tanggung jawab, menjadi alat yang kuat untuk melakukan analisis refactoring dari monolith ke microservices dan untuk menilai dampak perubahan terhadap batas antarmuka dan dependensi lalu lintas data. Untuk penelitian ini, pemanfaatan C4 memudahkan dokumentasi transformasi arsitektural yang diperlukan untuk membuat vertical slice pada kedua arsitektur sehingga perbandingan menjadi traceable.



Gambar 2.2. Ilustrasi C4 Context diagram untuk sistem Point of Sale yang menunjukkan aktor eksternal (kasir, sistem pembayaran, laporan akuntansi) dan batas sistem POS)

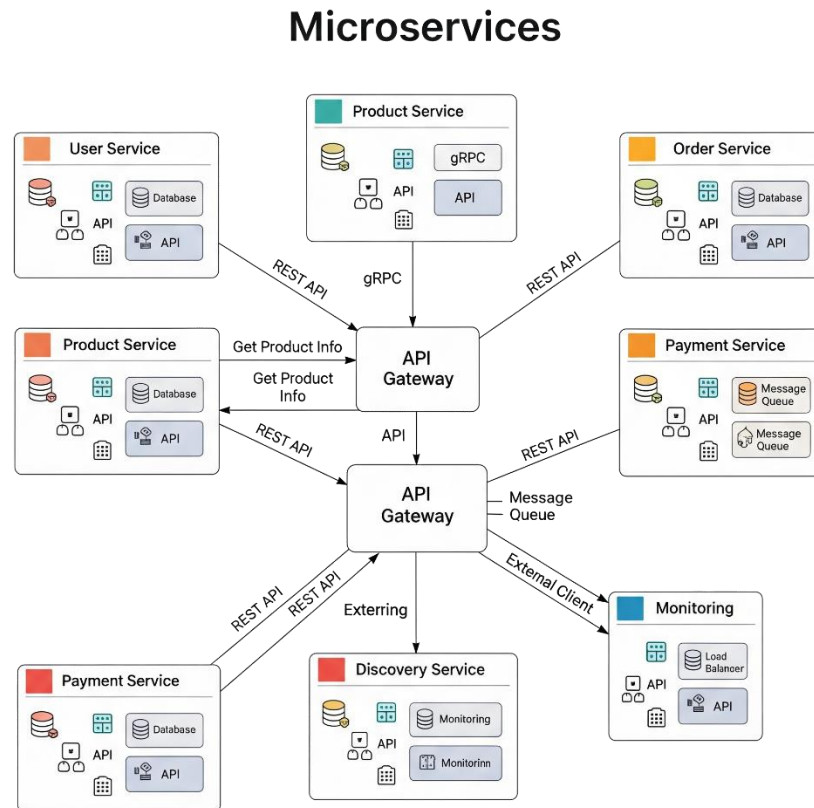
Secara praktis, model C4 mempermudah penempatan titik pengukuran metrik kinerja dan observabilitas karena setiap container dan komponen diidentifikasi secara eksplisit. Taibi dan Systä (2020) mengusulkan kerangka evaluasi berbasis dekomposisi yang menghubungkan metrik teknis dengan unit arsitektural, sehingga metrik seperti throughput dan latency dapat dikaitkan langsung ke container yang relevan. Dalam implementasi POS pada penelitian ini, diagram container C4 akan digunakan untuk menentukan di mana pengukuran Artillery harus diarahkan, misalnya menargetkan endpoint Transaction Service untuk mengukur latensi p99 transaksi, atau memantau Resource Utilization pada container Database Inventory untuk menghitung storage footprint. Pendekatan ini membuat proses pengukuran menjadi lebih terarah dan konsisten antara arsitektur pembanding.

Ketepatan C4 dalam menggambarkan antarmuka publik layanan memudahkan perencanaan evolusi API dan refaktorisasi tanpa merusak konsistensi integrasi antarlayanan. Stocker dan Zimmermann (2023) membahas bagaimana katalog pola refactoring API dan template dapat digunakan bersama representasi arsitektural untuk memandu restrukturisasi antarmuka. Dengan menempatkan definisi API dan dependensi lintas-container pada diagram C4, tim dapat merencanakan skenario backward compatibility dan penggantian implementasi secara terkontrol, faktor yang sangat penting ketika menilai trade-off antara monolith dan microservices pada sistem POS yang harus menjaga interoperabilitasnya terhadap sistem pembayaran eksternal (Stocker & Zimmermann, 2023).

Secara metodologis, pedoman praktis untuk menyusun diagram C4 dalam penelitian ini adalah memulai dari diagram konteks yang menunjukkan bounded context DDD dan aktor eksternal, lalu menyusun container diagram untuk setiap bounded context yang dipilih sebagai vertical slice, dilanjutkan dengan diagram komponen untuk container utama seperti Transaction Service, dan terakhir dokumentasi komponen kunci hingga level kode bila diperlukan untuk analisis implementasi. (Kleppmann, 2017) mengingatkan pentingnya keterkaitan eksplisit antara desain arsitektural dan kebutuhan data ketika merancang sistem data-intensive, sehingga setiap artefak C4 yang dibuat harus menonjolkan aliran data dan titik kepemilikan data untuk mendukung evaluasi metrik seperti eventual consistency lag dan state rehydration time.

2.2 Arsitektur microservices

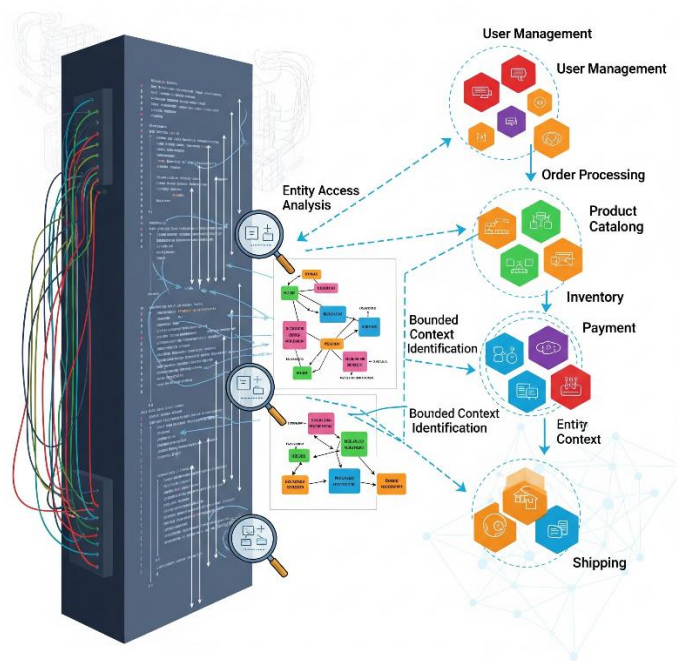
Arsitektur microservices didefinisikan sebagai pendekatan desain perangkat lunak yang memecah sistem monolitik menjadi sekumpulan layanan otonom yang masing-masing bertanggung jawab atas satu kapabilitas bisnis yang jelas. Tiap layanan berjalan dalam proses terpisah, berkomunikasi melalui antarmuka ringan, dan dapat dikembangkan serta dideploy secara independen. Konsekuensi praktisnya adalah siklus rilis yang lebih pendek untuk fitur tertentu, isolasi kegagalan yang lebih baik karena kesalahan pada satu layanan tidak langsung menjatuhkan seluruh sistem, serta kemampuan untuk memilih tumpukan teknologi yang paling sesuai per layanan. Pernyataan konsep dan karakteristik ini selaras dengan definisi operasional microservices (Lewis, James; Fowler, 2014)



Gambar 2.3. Diagram arsitektur microservices komponen yang menampilkan layanan-layanan otonom dan interaksi antarmuka mereka

Pendekatan microservices menjadi sangat relevan pada aplikasi yang bersifat data-intensive karena ketiga pilar kualitas yang diidentifikasi oleh Kleppmann, yakni reliabilitas, skalabilitas, dan maintainabilitas, dapat ditangani dengan pembagian tanggung jawab yang terisolasi. Dengan memecah beban kerja menjadi layanan-layanan domain-spesifik, beban baca atau tulis yang tinggi pada bagian tertentu dari sistem dapat ditangani dengan menskalakan hanya layanan terkait, sehingga sumber daya dapat diarahkan secara efisien tanpa perlu memperbesar seluruh aplikasi. Relevansi ini terlihat jelas pada sistem yang menuntut pemrosesan transaksi cepat, histori event yang panjang, dan kebutuhan audit; desain microservices menyediakan landasan untuk memenuhi kebutuhan tersebut dengan penyesuaian granular pada kapasitas dan konfigurasi layanan (Kleppmann, 2017).

Implementasi microservices yang efektif kerap dimulai dengan dekomposisi berdasar subdomain bisnis. Dekomposisi berdasarkan subdomain berarti memetakan bounded context domain-driven design ke layanan-layanan independen sehingga tiap layanan memiliki model domain, logika, dan batasan transaksional sendiri. Levezinho dan rekan menjelaskan pendekatan praktis untuk mengidentifikasi candidate decompositions dari monolit dengan menganalisis pola akses entitas dan hubungan bisnis, sehingga hasil dekomposisi menjadi lebih mudah divalidasi dan konsisten dengan perilaku aplikasi nyata. Pada konteks studi kasus Point of Sale, dekomposisi semacam ini memungkinkan memisahkan layanan transaksi penjualan, inventori, manajemen produk, dan otorisasi pembayaran sebagai unit-unit yang logis dan terukur, yang pada gilirannya mempermudah pengujian beban terfokus dan isolasi eksperimen (Levezinho et al., 2024).

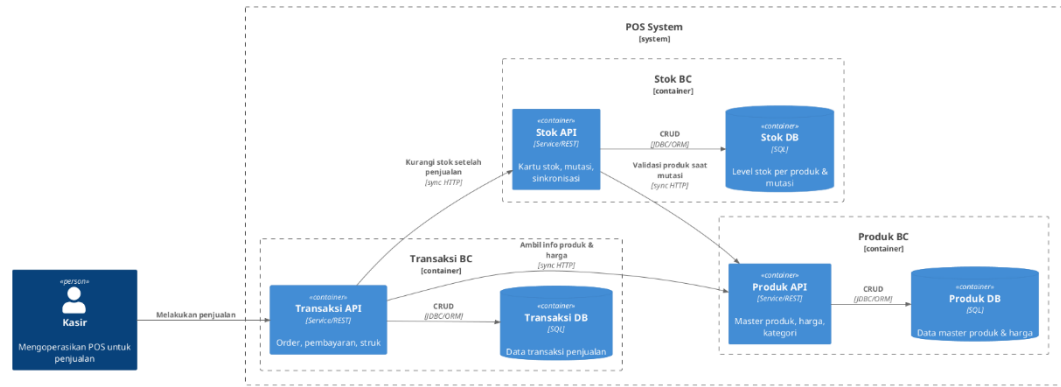


Gambar 2.4. Ilustrasi proses dekomposisi monolit menjadi layanan-layanan berdasarkan analisis akses entitas dan bounded context

Proses dekomposisi yang baik bukan hanya pemetaan fungsional, tetapi juga mempertimbangkan batasan transaksional dan konsistensi data. Kerangka evaluasi yang berorientasi metrik membantu menetapkan titik pemisahan layanan yang praktis. Taibi dan Systä menyarankan pendekatan metrik-basis yang mengkombinasikan analisis dependensi, ukuran tim, dan frekuensi perubahan modul untuk menghasilkan dekomposisi yang dapat dikelola (Taibi & Systä, 2020). Pendekatan ini relevan saat menyusun vertical slice untuk eksperimen karena memastikan bahwa setiap slice merepresentasikan alur fungsional end-to-end yang realistis namun tetap terukur sehingga metrik kinerja dan metrik developer dapat dikumpulkan secara konsisten.

Manajemen data dalam arsitektur microservices memunculkan pola polyglot persistence, yaitu kemampuan tiap layanan memilih model penyimpanan yang paling sesuai untuk kepentingan domainnya. Laigner dan kawan-kawan mendokumentasikan praktik-praktik nyata di industri yang menunjukkan bahwa pilihan model penyimpanan harus didorong oleh kebutuhan query, frekuensi update, dan skema evolusi, bukan sekadar by-default memilih satu jenis basis data. Pada sistem Point of Sale, misalnya, layanan katalog produk dapat memanfaatkan basis data dokumen agar mudah melakukan pencarian berbasis atribut, sedangkan layanan inventori dapat memilih basis data relasional untuk menjaga konsistensi kuantitas. Pilihan ini mempengaruhi

pola integrasi antar-layanan dan strategi transaksi lintas layanan sehingga harus menjadi bagian eksplisit dari rancangan eksperimen.



Gambar 2.5. Contoh pola polyglot persistence di mana layanan produk memakai dokumen store dan layanan inventori memakai relasional)

Arsitektur microservices sering dipadukan dengan pola asinkron dan event-driven untuk mengurangi ketergantungan pada transaksi terdistribusi yang berat. Chavan menelaah pola, jebakan, dan praktik terbaik event-driven dalam microservices dan menyatakan bahwa pemakaian event bus, publish-subscribe, serta event choreography yang terstruktur dapat menurunkan coupling sinkron. Untuk riset ini, relevansi pendekatan event-driven adalah bahwa eksperimen dapat mengukur bagaimana mekanisme penerbitan event, fan-out, dan eventual consistency memengaruhi metrik latensi baca, konsistensi data, dan kompleksitas implementasi pada vertical slice POS. Implementasi event-driven yang baik juga memberikan mekanisme alami untuk integrasi dengan Event Sourcing dan CQRS.

Desain microservices menghadirkan trade-off operasional yang nyata. (Vale et al., 2022) melakukan studi empiris pada pilihan pola microservices dan menemukan bahwa setiap pola yang diadopsi memberikan keuntungan tertentu tetapi juga menambah overhead lain, seperti kebutuhan orkestrasi, monitoring, dan refactoring API. Untuk studi perbandingan antara arsitektur hybrid dan monolith, temuan ini penting karena eksperimen harus memasukkan metrik yang menangkap biaya koordinasi, misalnya jumlah panggilan jaringan antar-layanan, latensi puncak akibat ketergantungan layanan, dan kompleksitas skema API yang harus dipertahankan. Rancangan evaluasi harus secara eksplisit mengukur overhead ini agar hasil komparatif tidak bias hanya pada metrik throughput.

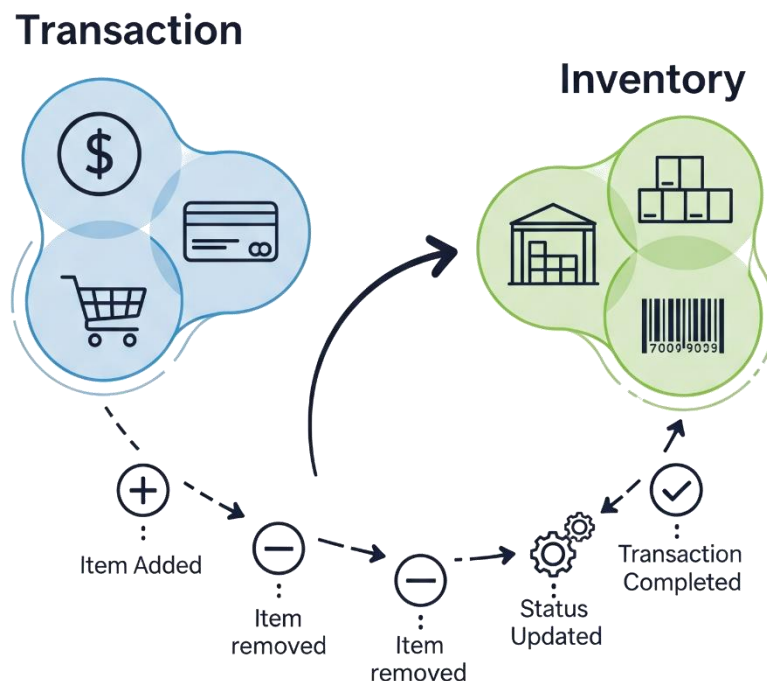
2.3 Domain-Driven Design (DDD)

Domain-Driven Design adalah pendekatan rekayasa perangkat lunak yang menempatkan model domain bisnis sebagai pusat perancangan sistem. Pendekatan ini menekankan penggunaan bahasa yang sama antara pemangku kepentingan bisnis dan tim teknis, yang disebut *ubiquitous language*, sehingga logika domain diterjemahkan secara eksplisit ke dalam model perangkat lunak. Tujuan penerapan DDD pada konteks *microservices* adalah memetakan konsep bisnis ke unit-unit modular yang bermakna, sehingga struktur perangkat lunak mencerminkan struktur domain organisasi. Dengan fondasi ini, DDD tidak sekadar teknik pemodelan, tetapi menjadi kerangka kerja untuk meminimalkan ambiguitas persyaratan dan menjaga koherensi model ketika sistem berevolusi (Hippchen et al., 2017).

DDD menyediakan konsep *bounded context* yang kritis untuk dekomposisi layanan. *Bounded context* menentukan batas di mana model tertentu berlaku secara konsisten, sehingga istilah dan aturan dalam satu konteks tidak harus berlaku di konteks lain. (Levezinho et al., 2024) menggambarkan bagaimana pemetaan kandidat dekomposisi monolitik ke *bounded context* yang diidentifikasi dari pola akses entitas dapat membantu proses migrasi dan desain layanan. Dalam praktiknya, analisis akses entitas dan interaksi antar-komponen pada sistem lama memungkinkan identifikasi *boundary* yang alami, yang pada gilirannya meminimalkan korelasi data dan interdependensi antar-layanan. Pendekatan ini memberi prosedur yang dapat diulang untuk menentukan layanan-layanan awal pada fase desain, bukan sekadar pembagian fungsional yang bersifat teoretis.

Dalam konteks industri, penerapan DDD pada *microservices* sering didorong oleh kebutuhan nyata akan *maintainability* dan *deployment agility*. (Nogueira et al., 2024) mengamati bahwa organisasi yang mengadopsi *microservices* dan DDD melaporkan peningkatan kecepatan pengiriman fitur dan penurunan regresi, namun hal tersebut memerlukan disiplin pada pemodelan domain dan pengujian. Untuk studi kasus POS, temuan ini relevan karena POS menuntut evolusi fitur yang cepat, integrasi dengan banyak subsistem, serta stabilitas operasional. Oleh karena itu, adopsi DDD pada rancangan POS membantu memastikan bahwa perubahan bisnis diterjemahkan ke unit layanan yang terdefinisi dengan jelas, sehingga tim dapat melakukan iterasi tanpa merusak integritas keseluruhan sistem.

Konsep agregat dalam DDD menetapkan batas konsistensi internal dan aturan transaksi pada unit domain yang koheren. Praktik menggunakan agregat yang tepat memungkinkan integrasi yang aman antara pola Event Sourcing dan CQRS, karena setiap agregat menjadi unit yang menghasilkan event yang merepresentasikan perubahan state. Pada POS, misalnya, agregat Transaction mengontrol invariants terkait penulisan transaksi dan garis besar aturan bisnis yang harus dipulihkan secara atomik, sementara agregat Inventory mengendalikan kuantitas barang dan aturan reservasi. Dengan menempatkan konsistensi kuat di dalam agregat dan membiarkan konsistensi antar-agregat bersifat eventual, arsitek dapat menyeimbangkan kebutuhan konsistensi dan skalabilitas (N'guessan et al., 2023).



Gambar 2.6. Ilustrasi agregat Transaction dan Inventory serta jalur event yang dihasilkan oleh perubahan pada agregat

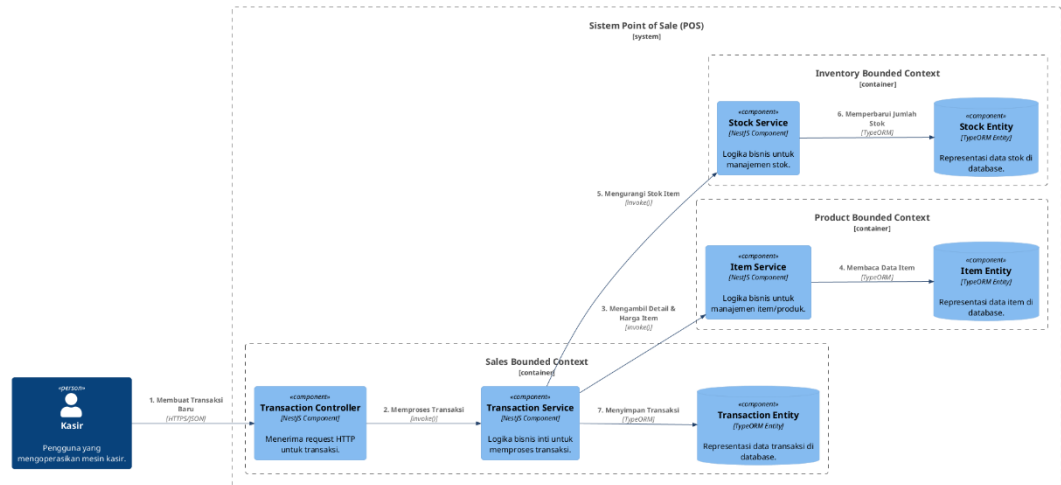
Pemecahan domain yang mengikuti prinsip DDD juga memberikan pijakan kuat untuk menyusun tim developer menurut bounded context. (Taibi & Systä, 2020) menguraikan kerangka evaluasi dekomposisi yang mempertimbangkan metrik organisasi, sehingga pemisahan teknis sejalan dengan struktur tim. Ketika tim bertanggung jawab penuh terhadap satu bounded context, mereka memiliki kepemilikan kode, data, dan pipeline deployment, yang mengurangi latensi koordinasi

dan meningkatkan kepemilikan kualitas. Untuk penelitian ini, pengukuran metrik pengalaman developer seperti lead time for changes dan context-switching count akan menunjukkan apakah struktur tim yang mengikuti bounded context memberikan keuntungan operasional nyata pada skenario POS.

Praktik pemetaan konteks dan pembuatan context map adalah langkah penting untuk mengelola interaksi antar-bounded context, termasuk pola integrasi yang dipilih seperti shared kernel, anti-corruption layer, atau published language. (Stocker & Zimmermann, 2023) menekankan bahwa evolusi antarmuka memerlukan strategi refaktorisasi API yang sistematis agar layanan tetap dapat berinteraksi tanpa memicu kerumitan interoperabilitas. Pada kasus POS, peta konteks membantu menentukan jalur integrasi antara layanan pembayaran dan layanan inventori, serta memandu pemilihan pola komunikasi sinkron atau asinkron berdasarkan karakteristik operasi. Context map juga menjadi dokumen referensi bagi pengujian integrasi dan pengawasan konsistensi domain antar-layanan.

Pemanfaatan DDD memengaruhi keputusan data ownership dan pola persistence. Laigner dan kolega mendokumentasikan praktik-praktik manajemen data pada microservices dan menekankan bahwa kepemilikan data per-layanan mengurangi kebutuhan transaksi terdistribusi dan memberikan kebebasan untuk memilih model penyimpanan yang paling sesuai. Dalam POS, menerapkan DDD memungkinkan layanan yang intensif baca memilih penyimpanan terdenormalisasi yang dioptimalkan untuk query cepat, sementara layanan yang membutuhkan akurasi penulisan tinggi dapat memilih penyimpanan yang mendukung konsistensi. Pendekatan ini selaras dengan tujuan penelitian untuk mengukur perbedaan kinerja dan footprint penyimpanan antar-layanan yang didekomposisi menurut DDD (Laigner et al., 2021).

Pada aspek migrasi dari monolith ke layanan berdasar DDD, metodologi praktis diperlukan untuk mengidentifikasi candidate slices yang layak dipecah. Vähä pada studi kasus backend permainan daring dan (Rahmatulloh et al., 2021) pada aplikasi IoT menunjukkan bahwa pendekatan bertahap, dimulai dari pemetaan domain dan pemisahan vertical slice yang representatif, mengurangi risiko gangguan layanan. Untuk POS, strategi serupa direkomendasikan: mulai dengan memisahkan vertical slice yang melibatkan alur lengkap transaksi dari input hingga pencatatan dan pelaporan. Dengan demikian penelitian ini akan mengimplementasikan vertical slice berbasis bounded context sebagai unit uji, sehingga hasil perbandingan antara microservices dan monolith menjadi komparatif dan dapat diukur.



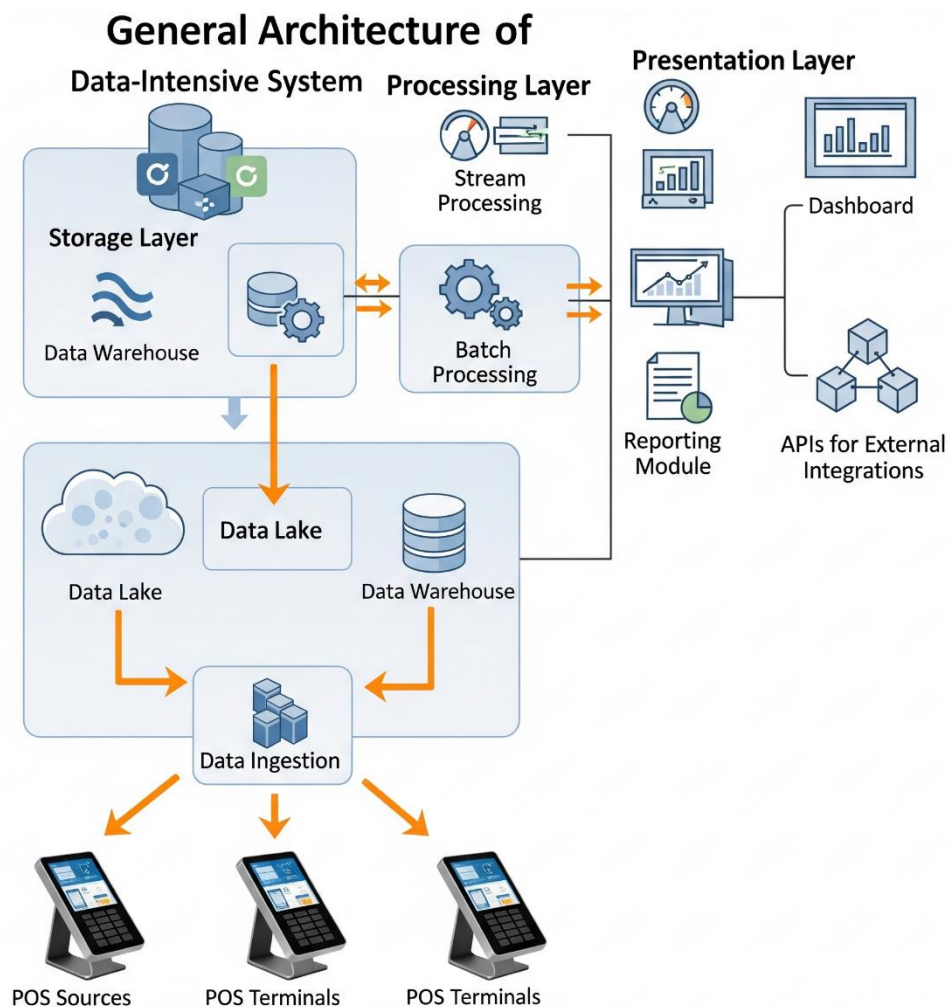
Gambar 2.7. Diagram vertical slice untuk POS yang menunjukkan pemisahan end-to-end dalam satu bounded context

Secara sintesis, DDD memberikan keuntungan strategis untuk sistem POS yang bersifat data-intensive karena ia menyediakan mekanisme teoritis dan praktis untuk menentukan boundary layanan yang meaningful, mengarahkan pilihan teknologi pelayanan, dan menyelaraskan struktur tim dengan arsitektur teknis. Kleppmann menegaskan bahwa ketika data menjadi sumber tantangan utama, arsitektur yang eksplisit pada domain business responsibilities, ownership, dan model data memberikan landasan yang lebih kuat untuk mencapai reliabilitas, skalabilitas, dan maintainability. Oleh karena itu, penerapan DDD pada penelitian ini bukan sekadar preferensi desain, tetapi sebagai pendekatan metodologis yang memungkinkan evaluasi multi-dimensi arsitektur menjadi terukur dan relevan terhadap kebutuhan industri. (Referensi: Kleppmann, 2017)

2.4 Manajemen Data pada Sistem Data-Intensif

Manajemen data pada sistem data-intensif merupakan fondasi yang menentukan keandalan, skalabilitas, dan kemudahan pemeliharaan suatu aplikasi. Sistem data-intensif mengacu pada aplikasi yang beban utamanya terletak pada volume, variasi, dan kecepatan data, sehingga tantangan utama bukan hanya pada komputasi, tetapi pada pengelolaan aliran, penyimpanan, serta konsistensi data di seluruh komponen sistem. Kleppmann (2017) menekankan bahwa strategi manajemen data harus mempertimbangkan sifat sistem yang berubah secara dinamis, termasuk

skema data yang berevolusi, kebutuhan akan toleransi kesalahan, dan kemampuan untuk melakukan rehidrasi state dari sumber data yang tersebar.

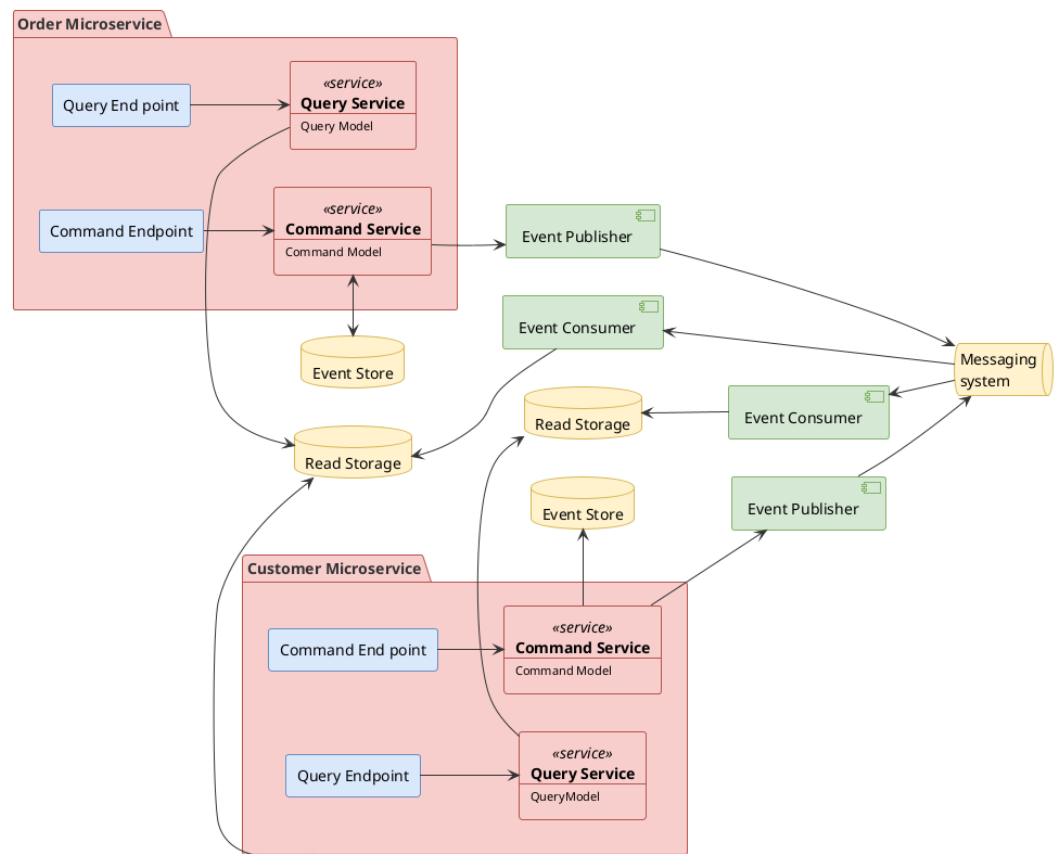


Gambar 2.8. Ilustrasi arsitektur umum sistem data-intensif dan aliran datanya

Pendekatan tradisional yang mengandalkan basis data terpusat dalam arsitektur monolitik seringkali menghadapi bottleneck pada skala besar, terutama ketika terjadi lonjakan beban transaksi. Hal ini menyebabkan peningkatan latensi, penurunan throughput, dan tingginya risiko downtime jika terjadi kegagalan tunggal. Dalam konteks aplikasi Point of Sale (POS) yang bersifat data-intensif, manajemen data yang efektif harus memastikan bahwa data transaksi dapat diproses secara cepat, tersedia hampir secara real-time, dan tetap konsisten walaupun sistem terdistribusi. Oleh karena

itu, strategi manajemen data pada penelitian ini lebih condong ke pendekatan yang memanfaatkan kombinasi arsitektur microservices, CQRS, dan Event Sourcing, yang memungkinkan isolasi tanggung jawab data sekaligus fleksibilitas dalam evolusi skema.

Dalam microservices tantangan manajemen data terletak pada tiga aspek utama, konsistensi data antar layanan, sinkronisasi skema antar versi, dan optimisasi performa query lintas layanan. Penggunaan database terpisah per layanan memang meningkatkan otonomi, namun menambah kompleksitas dalam menjaga integritas data (Taibi & Systä, 2020). Pendekatan seperti Event Sourcing mampu mengatasi sebagian tantangan ini dengan cara menyimpan semua perubahan state sebagai rangkaian peristiwa (events) yang tidak dapat diubah, sehingga memungkinkan rekonstruksi state di titik waktu manapun. Dengan demikian, jika terjadi inkonsistensi, sistem dapat melakukan rehidrasi ulang state dari event log tanpa bergantung pada satu sumber data tunggal.



Gambar 2.9. Contoh alur data pada arsitektur microservices dengan Event Sourcing dan CQRS

Pendekatan ini relevan dengan kasus penelitian karena aplikasi POS yang diuji memiliki karakteristik transaksi tinggi, data yang bersifat append-only, dan kebutuhan historis data yang kuat untuk pelacakan audit. Dengan memisahkan beban baca dan tulis melalui CQRS, query dapat dioptimalkan secara terpisah dari operasi penulisan. Hal ini berdampak pada peningkatan throughput, penurunan latensi pada puncak beban, dan pengurangan risiko deadlock yang umum terjadi pada basis data monolitik.

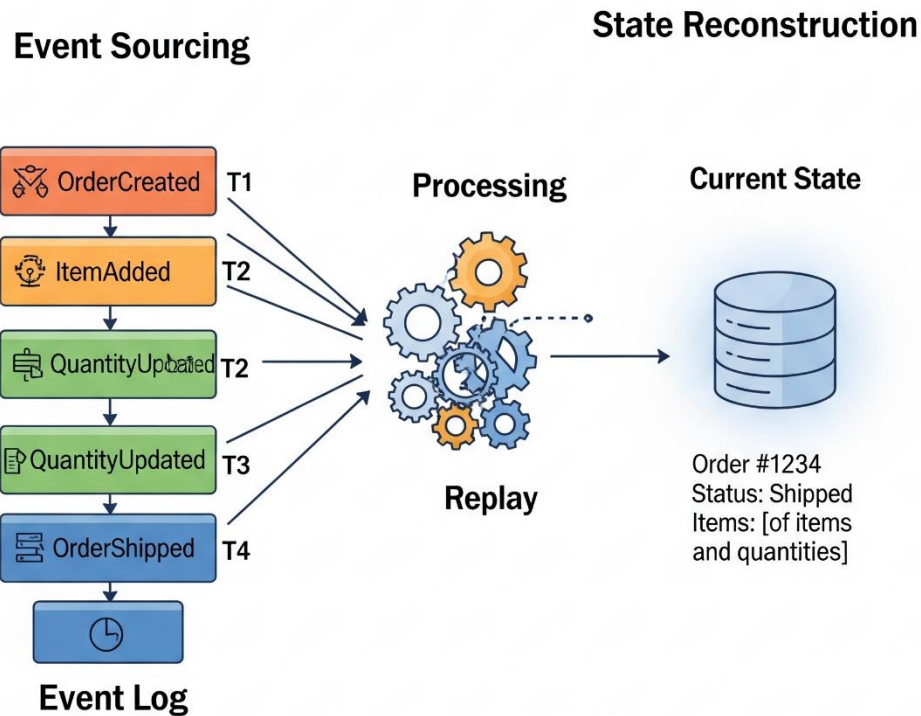
Selain itu, strategi manajemen data pada sistem data-intensif modern juga memerlukan kemampuan elastisitas dan skalabilitas horizontal. Volume data yang besar, lapisan penyimpanan harus dirancang untuk menambahkan node baru tanpa mengganggu operasional sistem. Dalam kasus ini, event store dapat diimplementasikan pada sistem log terdistribusi seperti Apache Kafka atau database append-only yang mendukung replikasi partisi, sehingga kapasitas dapat ditingkatkan seiring pertumbuhan data (Kleppmann, 2017). Keterkaitan langsung pendekatan ini dengan penelitian adalah pada kemampuannya untuk mengakomodasi pengujian multi-dimensi yang direncanakan, baik dari sisi kinerja teknis seperti throughput dan latensi, maupun dari sisi arsitektural seperti eventual consistency lag dan state rehydration time. Manajemen data yang terdesain baik pada sistem data-intensif memungkinkan eksperimen dilakukan secara terukur di lingkungan staging yang menyerupai kondisi produksi tanpa risiko merusak sistem aktual.

2.5 Event Sourcing

Event Sourcing adalah pendekatan penyimpanan data di mana setiap perubahan terhadap state aplikasi direkam sebagai serangkaian event yang tidak dapat diubah. Alih-alih hanya menyimpan state akhir dalam basis data, sistem mempertahankan log lengkap dari semua event yang telah terjadi, yang kemudian dapat diputar ulang untuk merekonstruksi state terkini. Fowler (2005) menjelaskan bahwa prinsip utama dari Event Sourcing adalah “state as a result of events”, di mana sumber kebenaran sistem bukanlah tabel data, melainkan riwayat peristiwa itu sendiri. Setiap event bersifat immutable sehingga dapat dijadikan arsip historis yang akurat sekaligus sumber analisis yang kaya.

Pendekatan ini menjadi sangat relevan untuk sistem data-intensive seperti backend Point of Sale (POS) karena transaksi yang terjadi biasanya bersifat audit-critical. Setiap perubahan stok, harga, atau status transaksi perlu dapat dilacak kembali ke titik waktu spesifik. Dengan Event Sourcing, setiap mutasi data memiliki catatan

terstruktur yang dapat direkonstruksi tanpa kehilangan informasi historis. Hal ini memungkinkan verifikasi integritas data yang lebih kuat dibandingkan pendekatan konvensional yang hanya menyimpan nilai terkini.



Gambar 2.10. Ilustrasi konsep dasar Event Sourcing: dari event log ke state rekonstruksi

(Laigner et al., 2021) menguraikan bahwa salah satu kekuatan utama Event Sourcing adalah kemampuannya menangani schema evolution secara lebih fleksibel. Karena data utama disimpan dalam bentuk event, perubahan struktur data di masa depan dapat dikelola melalui mekanisme event upcasting, di mana event lama diinterpretasikan ulang dengan format baru saat diperlukan. Ini sangat berguna dalam lingkungan pengembangan yang iteratif seperti pada arsitektur Microservices, di mana model domain dapat berubah seiring berjalannya waktu. Selain itu, rekonstruksi state dari event log membuat pengujian dan debugging menjadi lebih terkontrol karena developer dapat memutar ulang skenario spesifik dari masa lalu untuk menganalisis bug atau anomali.

Dalam konteks kombinasi arsitektur Microservices, CQRS, dan Event Sourcing seperti yang diimplementasikan pada penelitian ini, manfaatnya semakin nyata.

Menurut penelitian (Fansha et al., 2021), integrasi Event Sourcing dengan CQRS meningkatkan skalabilitas karena beban baca dan tulis dapat dioptimalkan secara terpisah. Event Sourcing menyediakan aliran event yang konsisten untuk *read model*, sementara *write model* fokus pada validasi dan persistensi event baru. Ini mengurangi bottleneck pada basis data terpusat dan memaksimalkan throughput pada sistem data-intensive.

Lebih jauh lagi, dengan volume transaksi tinggi Event Sourcing memberikan ketahanan terhadap kehilangan data karena event log dapat direplikasi secara asinkron ke node lain tanpa harus melakukan sinkronisasi penuh pada state akhir (Laigner et al., 2021). Keuntungan ini sangat krusial untuk POS yang memerlukan ketersediaan tinggi, bahkan ketika salah satu node mengalami kegagalan. Dengan demikian, Event Sourcing tidak hanya menawarkan transparansi historis, tetapi juga meningkatkan keandalan dan *fault tolerance* pada sistem.

Pendekatan ini lebih unggul dibanding penyimpanan state langsung pada monolith tradisional, yang sering kali mengalami kesulitan dalam mengaudit perubahan secara granular atau memutar ulang kondisi masa lalu. Event Sourcing memisahkan aliran mutasi data dari representasi state sehingga memungkinkan fleksibilitas dalam mengonsumsi dan menginterpretasikan data. Dalam kasus penelitian ini, di mana arsitektur monolith digunakan sebagai pembanding, penerapan Event Sourcing pada Microservices diproyeksikan akan menunjukkan keunggulan dalam aspek state rehydration time, eventual consistency lag, dan efisiensi penggunaan storage, karena log event dapat dioptimalkan melalui teknik kompresi atau snapshotting.

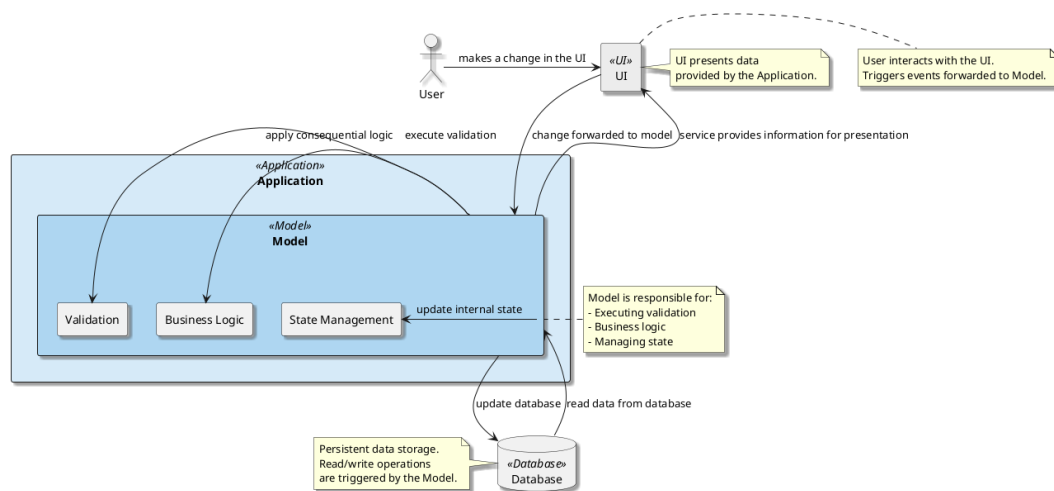
Dengan sifatnya yang immutable, rekonstruktif, dan terdistribusi dengan baik, Event Sourcing menjadi fondasi yang tepat untuk membangun backend POS data-intensive yang aman, dapat diaudit, dan mampu beradaptasi terhadap perubahan domain model di masa depan.

2.6 Command Query Responsibility Segregation (CQRS)

Command Query Responsibility Segregation (CQRS) adalah sebuah pola arsitektur yang memisahkan jalur penanganan perintah (command) dan kueri (query) pada sistem perangkat lunak. Inti dari konsep ini adalah bahwa operasi yang memodifikasi data (command) dan operasi yang membaca data (query) tidak menggunakan model data yang sama. Pemisahan ini bertujuan untuk mengoptimalkan

kinerja, skalabilitas, dan keamanan sistem, terutama pada aplikasi berskala besar dan data-intensif. CQRS pertama kali dipopulerkan oleh Greg Young, dan banyak digunakan dalam arsitektur modern yang berbasis microservices dan event-driven (Lewis, James; Fowler, 2014).

Pola ini muncul dari pengamatan bahwa kebutuhan untuk membaca dan menulis data sering kali memiliki karakteristik teknis yang berbeda. Proses menulis data cenderung lebih kompleks karena melibatkan validasi bisnis, pengelolaan transaksi, dan penerapan aturan konsistensi. Sebaliknya, proses membaca data lebih fokus pada kecepatan pengambilan informasi, pemetaan data ke bentuk presentasi, dan pengoptimalan query agar cepat diakses oleh pengguna. Dengan memisahkan kedua proses ini, CQRS meminimalkan kompromi desain yang biasanya terjadi ketika satu model harus memenuhi kedua kebutuhan tersebut.

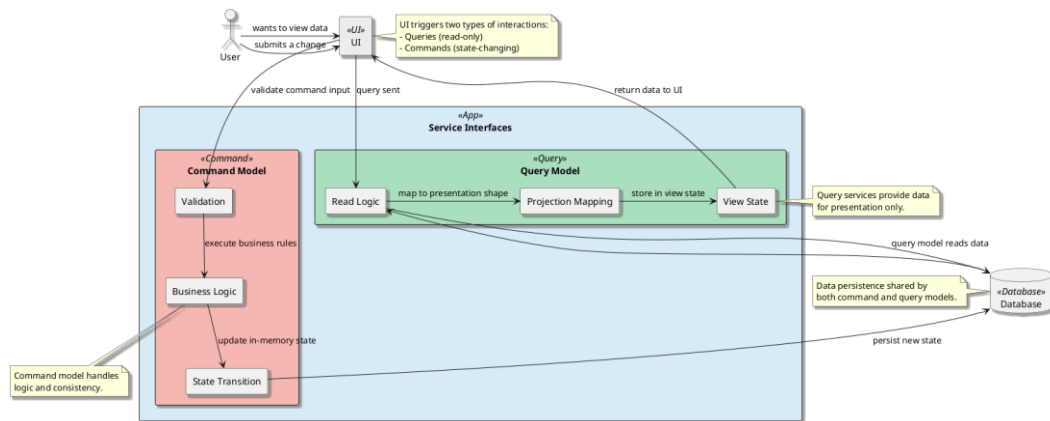


Gambar 2.11. Arsitektur tanpa penerapan CQRS

Pada arsitektur tradisional tanpa CQRS, satu model digunakan untuk menangani perintah dan kueri sekaligus. Pendekatan ini memang sederhana untuk implementasi awal, namun ketika sistem tumbuh, model ini akan menghadapi hambatan kinerja dan kompleksitas yang tinggi. Hal ini terjadi karena setiap perubahan pada struktur data harus mempertimbangkan kedua kebutuhan baca dan tulis secara bersamaan. Sebagai contoh, menambahkan optimasi query yang kompleks sering kali mengganggu logika penulisan data, atau sebaliknya, modifikasi validasi data dapat mempengaruhi performa query.

Penerapan CQRS memisahkan kedua model tersebut. Model command digunakan khusus untuk menangani perintah yang mengubah state sistem, sedangkan

model query hanya bertugas mengakses dan menyajikan data. Dalam implementasinya, model command umumnya terintegrasi erat dengan domain model dan validasi bisnis, sedangkan model query lebih ringan dan sering kali dioptimalkan dengan denormalisasi atau cache untuk meningkatkan kecepatan akses (Ataei & Staegemann, 2023).



Gambar 2.12. Arsitektur dengan penerapan CQRS

Dengan pemisahan ini, CQRS memberikan sejumlah keuntungan signifikan. Pertama, pemeliharaan dan pengembangan fitur menjadi lebih terfokus karena tim dapat mengubah logika bisnis tanpa harus khawatir mempengaruhi performa query. Kedua, skalabilitas sistem meningkat karena command dan query dapat diskalakan secara independen sesuai kebutuhan beban kerja. Ketiga, integrasi dengan Event Sourcing menjadi lebih mudah, di mana setiap perubahan state dapat direkam sebagai rangkaian event yang dapat di-replay untuk keperluan audit, debugging, atau rekonstruksi state (Laigner et al., 2021).

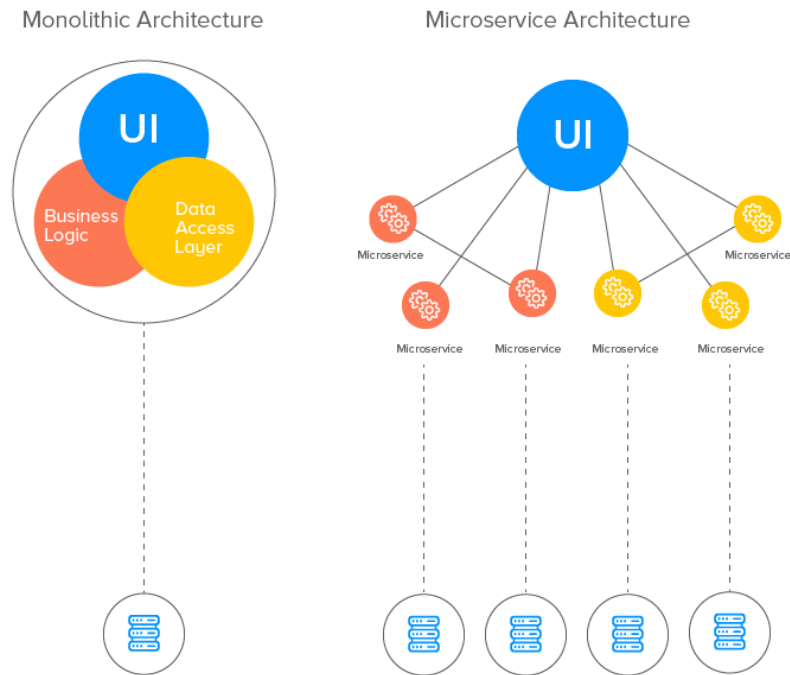
CQRS juga memberikan manfaat dalam konteks sistem yang memerlukan konsistensi eventual. Karena command dan query dipisahkan, data untuk keperluan baca dapat dioptimalkan tanpa mengganggu jalur pemrosesan tulis, meskipun hal ini mungkin mengakibatkan adanya jeda waktu singkat antara update data dan ketersediaannya di query model. Dalam praktiknya, pendekatan ini relevan untuk sistem yang bersifat data-intensif dengan jumlah transaksi tinggi, seperti platform e-commerce, sistem perbankan, dan sistem IoT skala besar (Kabbedijk et al., 2012).

Dalam kasus yang sedang dibahas, penerapan CQRS menjadi sangat relevan karena sistem memerlukan kemampuan untuk menangani beban baca dan tulis yang

tinggi secara bersamaan, dengan kebutuhan optimasi query yang berbeda dari proses penulisan data. Model command dapat difokuskan pada penerapan validasi bisnis yang ketat dan pengelolaan konsistensi data, sementara model query dapat dioptimalkan untuk pengambilan data cepat dan efisien, termasuk melalui penggunaan database terpisah atau cache. Dengan demikian, CQRS tidak hanya meningkatkan performa, tetapi juga mempermudah evolusi sistem seiring pertumbuhan kompleksitasnya.

2.7 Perbandingan Microservices dan Monolith

Perbandingan antara arsitektur microservices dan monolith menjadi hal yang krusial dalam proses pemilihan pendekatan pengembangan aplikasi yang tepat, terutama untuk sistem berskala besar dan memiliki intensitas data yang tinggi. Arsitektur monolith secara tradisional menggabungkan seluruh fungsionalitas aplikasi ke dalam satu basis kode yang saling terintegrasi, sehingga setiap perubahan atau pembaruan modul memerlukan proses build dan deploy ulang secara keseluruhan. Sebaliknya, microservices membagi fungsionalitas aplikasi menjadi layanan-layanan kecil yang berdiri sendiri dan dapat dikembangkan, di-deploy, serta diskalakan secara independent (Lewis, James; Fowler, 2014), perbedaan mendasar ini berdampak langsung terhadap fleksibilitas, kinerja tim pengembang, serta kemampuan sistem untuk beradaptasi dengan perubahan kebutuhan bisnis.



Gambar 2.13. Perbedaan struktur arsitektur monolith dan microservices

Salah satu keunggulan microservices adalah kemampuannya dalam mengadopsi teknologi yang berbeda pada setiap layanan. Hal ini sangat relevan bagi sistem yang memerlukan integrasi beragam komponen, seperti pada kasus pengolahan data secara real-time atau ketika menghadapi kebutuhan komputasi yang bervariasi. Sebaliknya, monolith cenderung membatasi developer pada satu tumpukan teknologi karena keterpaduan kode yang tinggi. (Rahmatulloh et al., 2021) menunjukkan bahwa dalam pengujian performa berbasis Apache Kafka, arsitektur microservices dapat memproses data dalam throughput yang lebih stabil saat volume data meningkat, sedangkan monolith mengalami penurunan performa signifikan pada skenario beban berat.

Selain faktor kinerja, perbedaan juga terlihat pada aspek pengelolaan siklus hidup aplikasi. Dalam microservices, siklus pengembangan dapat berjalan paralel karena setiap tim bertanggung jawab pada satu atau beberapa layanan tertentu. Pendekatan ini mengurangi ketergantungan antar tim dan mempercepat penerapan perubahan. Sebaliknya, monolith menuntut koordinasi intensif dan penyesuaian lintas modul, yang pada akhirnya memperpanjang waktu rilis. Microservices memberikan trade-off positif pada dimensi maintainability dan scalability, walaupun di sisi lain

memperkenalkan kompleksitas tambahan pada manajemen deployment, komunikasi antar layanan, dan pengelolaan data terdistribusi (Overeem et al., 2021).

Pada sisi skalabilitas, microservices memungkinkan skala vertikal dan horizontal pada layanan tertentu yang mengalami lonjakan beban, tanpa perlu menambah kapasitas pada seluruh sistem. Pendekatan ini terbukti efisien untuk kasus dengan distribusi beban yang tidak merata, misalnya hanya modul analitik yang memerlukan kapasitas komputasi tambahan. Sebaliknya, monolith memaksa peningkatan kapasitas secara keseluruhan meskipun hanya sebagian kecil fungsionalitas yang membutuhkan sumber daya tambahan. (Ashwin Chavan, 2021) menyoroti bahwa keunggulan ini membuat microservices lebih sesuai untuk sistem yang dinamis dan membutuhkan respon cepat terhadap lonjakan trafik atau perubahan permintaan.

Dalam konteks penelitian ini, perbandingan microservices dan monolith menjadi relevan karena sistem yang diuji menggabungkan pendekatan microservices dengan CQRS dan event sourcing untuk mencapai kinerja optimal. Pemisahan layanan memungkinkan pengujian dan optimisasi yang lebih fokus pada bagian tertentu, misalnya modul penanganan perintah (command handling) dan modul query yang masing-masing memiliki kebutuhan sumber daya berbeda. Pendekatan ini tidak hanya meningkatkan kinerja, tetapi juga mempermudah pengendalian kompleksitas seiring bertambahnya fitur dan volume data.

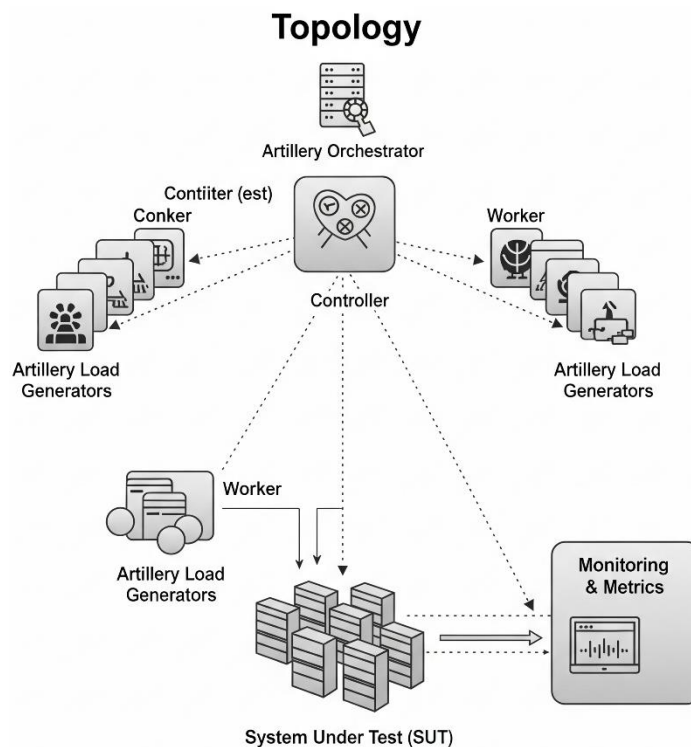
2.8 Pengujian Dimensi Teknis & Kinerja Aplikasi

Pengujian dimensi teknis dan kinerja adalah fondasi untuk menilai apakah sebuah arsitektur mampu memenuhi kebutuhan reliabilitas, skalabilitas, dan maintainability yang diuraikan pada pendahuluan. Kleppmann menegaskan bahwa pada sistem data-intensive tujuan pengujian kinerja bukan sekadar menemukan angka puncak, melainkan memahami perilaku sistem di bawah beban nyata, pola kegagalan, dan batas di mana sifat non-fungsional mulai melanggar asumsi bisnis. Oleh karena itu, perancangan eksperimen kinerja harus dimulai dengan mendefinisikan skenario beban yang merepresentasikan aliran kerja dunia nyata pada POS puncak transaksi kasir, batch sinkronisasi laporan, dan beban baca untuk pencarian produk serta pembuatan laporan.

Metrik yang akan diukur harus dipilih secara cermat agar mampu menggambarkan aspek performa yang relevan. Taibi dan Systä mengusulkan kerangka

evaluasi berbasis dekomposisi yang mengaitkan metrik teknis ke unit arsitektural; dari perspektif ini, metrik inti meliputi throughput (ops/sec), latensi distribusi (p50, p95, p99), success rate (rasio request sukses terhadap gagal), serta penggunaan sumber daya (CPU dan memori) pada setiap container atau proses. Pengukuran latensi dalam persentil tinggi terutama penting untuk aplikasi POS karena nilai outlier p99 sering memengaruhi pengalaman pengguna dan SLA operasional.

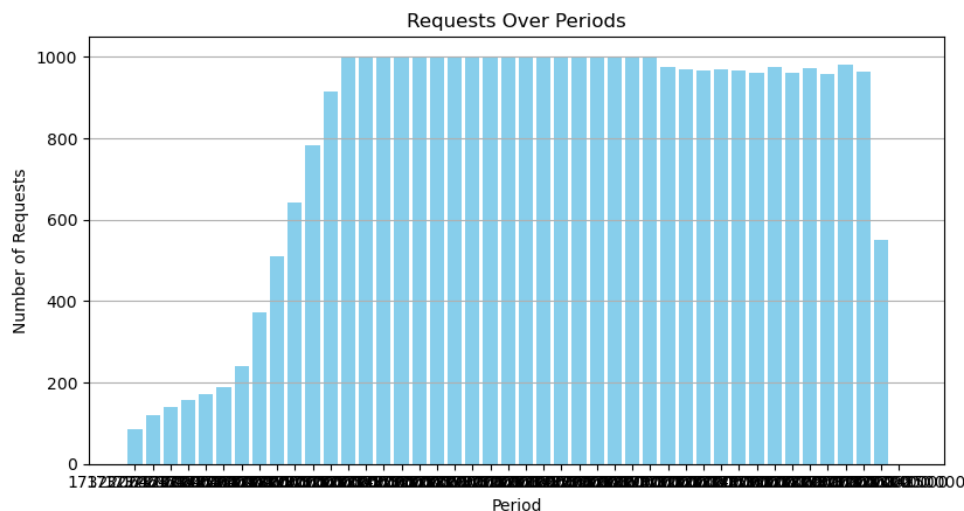
Desain eksperimen harus menjamin perbandingan yang adil antara arsitektur yang diuji. Praktik benchmarking yang baik menggunakan alat seperti Artillery buatlah profil beban yang konsisten (warm-up period, steady-state period, dan cool-down), jalankan beberapa iterasi uji untuk memperoleh variasi, dan isolasi variabel dengan menahan konfigurasi infrastruktur tetap sama untuk kedua arsitektur (misal CPU, memori, jaringan). Vertical slice yang diuji harus identik secara fungsional antara monolith dan microservices sehingga metrik merefleksikan perbedaan arsitektural, bukan perbedaan fungsional. Semua skrip Artillery, konfigurasi runtime, dan versi paket harus didokumentasikan agar eksperimen dapat direplikasi (Cornelius & Jaswal, 2024).



Gambar 2.14. Artillery topologi untuk benchmarking tools.

Praktik pengukuran harus memisahkan metrik sisi klien dan metrik sisi server untuk menghindari kesalahan interpretasi. (Fansha et al., 2021) yang melakukan load testing pada microservices dengan CQRS dan Event Sourcing, menyoroti pentingnya mengumpulkan metrik latensi end-to-end (apalagi p95/p99) dari sisi klien sekaligus memonitor resource utilization pada node backend, karena bottleneck yang dominan seringkali muncul di lapisan I/O, antrean event, atau pada subsystem penyimpanan event. Selain itu, pengukuran success rate harus memperhitungkan jenis kesalahan (timeout, 5xx, backpressure) dan mengukur kemampuan sistem untuk recover (misalnya retry otomatis atau replay event) selama dan setelah beban puncak.

Pengaturan beban uji harus mencakup skenario realistis yang merepresentasikan variasi beban POS steady transactional rate (day-to-day), burst (checkout rush), background reporting load, dan mixed workload yang menggabungkan baca intensif dengan tulis intensif. Christian dan Bisma menunjukkan bahwa pada perbandingan monolith versus microservices terutama ketika menggunakan messaging (seperti Kafka) performa relatif berubah tergantung pola beban: microservices sering unggul pada burst dan linear scaling, sementara monolith dapat unggul pada beban seragam dengan latensi rendah jika dioptimalkan. Oleh karena itu, set skenario harus dipilih agar mencerminkan kondisi operasional POS yang akan diuji (Christian & Bisma, 2021).



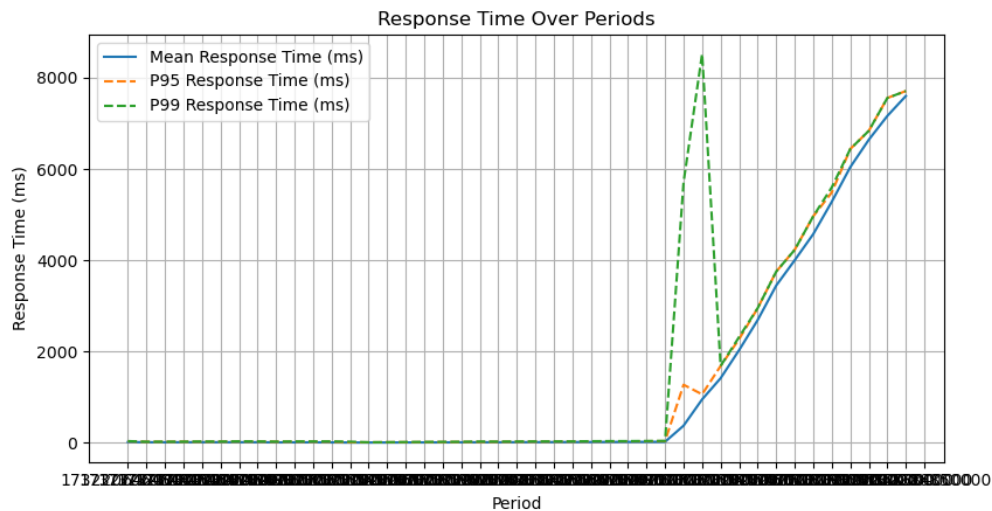
Gambar 2.15. Contoh profil beban Artillery.

Observabilitas dan instrumentasi adalah prasyarat untuk analisis yang bermakna. Manajemen data dan observabilitas di microservices menuntut pengumpulan metrik terperinci pada setiap layanan dan lapisan: metrik permintaan per endpoint, antrean event, ukuran backlog, waktu rehidrasi state, serta penggunaan disk dan I/O pada event store. Untuk keperluan penelitian ini, disarankan menempatkan endpoint metrik pada setiap container (misalnya exporter) dan mengumpulkan log terstruktur event timestamps agar dapat menghitung waktu dari command diterima hingga view model terbaru metrik penting untuk mengaitkan performa teknis dengan eventual consistency lag dan state rehydration time.

Rencana analisis hasil harus memasukkan prinsip statistik dasar menggunakan beberapa run independen untuk setiap skenario, hitung median dan persentil tingginya, sertakan interval kepercayaan pada pengukuran throughput dan latensi p95/p99, serta gunakan uji perbedaan statistik bila perlu untuk menilai signifikansi perbedaan antar-arsitektur. Interpretasi hasil harus memadukan metrik kuantitatif dengan konteks arsitektural misalnya, peningkatan throughput pada microservices mungkin datang dengan biaya CPU lebih tinggi atau storage footprint lebih besar; oleh karena itu laporan hasil harus menyajikan metrik performa bersamaan dengan metrik biaya operasional untuk menilai trade-off.

Agar pengujian juga menilai ketahanan operasional, eksperimen harus memasukkan skenario kegagalan terkontrol: pemadaman node, peningkatan latensi jaringan, atau pembuangan event (throttling). Arsitektur yang menggabungkan Event Sourcing dan CQRS memungkinkan pemulihan state yang lebih deterministik setelah insiden karena kemampuan replay event; pengujian harus mengukur recovery time objective (RTO) dan tingkat kehilangan transaksi saat failure injection dilakukan. Pada POS, skenario ini krusial karena kemampuan sistem untuk beroperasi atau pulih cepat berpengaruh langsung pada operasional toko.

Akhirnya, dokumentasi eksperimen dan reproducibility harus menjadi bagian integral dari laporan. Ataei dan Staegemann menekankan bahwa pada sistem big data, hasil pengujian kinerja hanya bermakna bila kondisi lingkungan dan konfigurasi tercatat lengkap: versi runtime, konfigurasi database, partisi event store, ukuran snapshot, dan parameter Artillery. Untuk penelitian ini, setiap eksperimen harus disertai artefak yang memungkinkan pengujian diulang oleh pihak lain: skrip load test, konfigurasi deployment (docker compose/k8s manifests), dan dataset sintetik yang merepresentasikan katalog produk dan pola transaksi POS. (Referensi: Ataei & Staegemann, 2023)



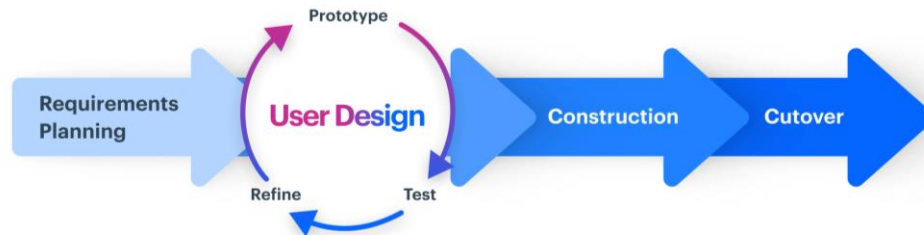
Gambar 2.16. Contoh hasil pengujian Artillery.

Dengan pendekatan pengujian yang terstruktur seperti di atas menghubungkan skenario realistis, metrik kuantitatif yang relevan, observabilitas lengkap, uji kegagalan, dan dokumentasi reproduktif penelitian akan mampu menghasilkan perbandingan teknis yang objektif antara kombinasi Microservices + CQRS + Event Sourcing dan arsitektur monolith pada konteks POS data-intensive. Hasil pengujian ini nantinya menjadi dasar empiris untuk mengevaluasi trade-off yang dijelaskan pada bab pendahuluan: apakah manfaat performa dan skalabilitas sepadan dengan biaya kompleksitas, storage footprint, dan overhead operasional.

2.9 Rapid Application Development (RAD)

Rapid Application Development (RAD) merupakan pendekatan pengembangan perangkat lunak yang menekankan kecepatan dalam menghasilkan prototipe dan iterasi sistem melalui keterlibatan intensif pengguna. Model ini mengedepankan fleksibilitas serta adaptasi cepat terhadap perubahan kebutuhan dibandingkan metodologi sekuensial tradisional. Proses utama dalam RAD biasanya mencakup empat fase: *requirements planning*, *user design*, *construction*, dan *cutover*. Keempat fase ini bersifat iteratif, memungkinkan sistem berkembang melalui prototyping cepat dan perbaikan berulang hingga sesuai dengan kebutuhan pengguna.

Rapid Application Development (RAD)



Gambar 2.17. RAD Development Life Cycle

Karakteristik utama RAD adalah siklus pengembangan yang singkat, keterlibatan pengguna yang intensif, serta pemanfaatan prototyping sebagai sarana komunikasi dan validasi kebutuhan. Hal ini menjadikan RAD cocok digunakan untuk proyek yang lingkungannya kompleks tetapi membutuhkan hasil yang cepat dan dapat diuji dalam tahap-tahap awal. Penerapan prinsip ini memastikan bahwa sistem tidak hanya memenuhi spesifikasi teknis, tetapi juga selaras dengan ekspektasi pemakai akhir.

Dibandingkan dengan metodologi Agile lainnya, RAD lebih menitikberatkan pada iterasi cepat dan penggunaan prototipe sebagai artefak komunikasi utama antara pengembang dan pengguna. Menurut hasil analisis mereka, RAD efektif diterapkan pada proyek akademik maupun sistem berskala terbatas karena meminimalkan overhead manajerial yang biasanya muncul dalam pendekatan Agile penuh (Khan et al., 2020).

Dalam konteks penelitian ini, penggunaan RAD memungkinkan proses implementasi arsitektur baik monolith maupun kombinasi Microservices, CQRS, dan Event Sourcing dapat dilakukan dalam unit-unit eksperimen yang lebih kecil dan terukur. Dengan memanfaatkan prototipe sebagai produk iteratif, setiap perbandingan arsitektur dapat dievaluasi secara cepat, baik dari dimensi teknis maupun dimensi pengembang. Hal ini selaras dengan tujuan penelitian yang menekankan evaluasi empiris melalui siklus eksperimen singkat, bukan hanya sekadar validasi konseptual.

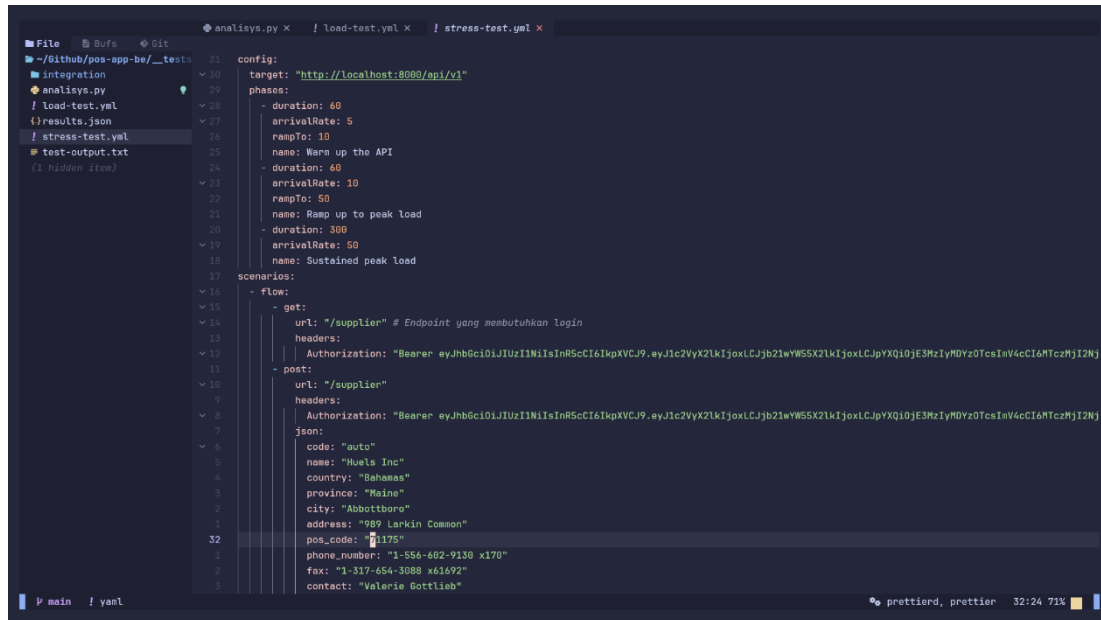
Selain itu, RAD juga mendukung pencatatan metrik pengembang seperti *lead time for changes* dan *context-switching count*, karena setiap iterasi proyek

terdokumentasi dengan jelas dari tahap desain hingga *cutover*. Dengan kata lain, model RAD memberikan struktur yang cukup ringan untuk memungkinkan eksperimen cepat, tetapi tetap sistematis dalam mengumpulkan data empiris yang dibutuhkan penelitian ini.

2.10 Pengujian Dimensi Arsitektural

Eventual consistency lag adalah jeda waktu antara saat sistem menerima perubahan (command) dan saat perubahan tersebut terlihat pada model baca (query model). Pada arsitektur yang memisahkan jalur tulis dan baca seperti CQRS yang program baca dan program ubahnya dipisah pengujian seperti ini sangat penting untuk melihat performa program karena saat suatu system melakukan operasi ubah mungkin system lainnya sedang melakukan operasi baca secara bersamaan.

Untuk mendapatkan data yang dapat dianalisis, pengukuran eventual consistency lag perlu diinstrumentasi menjadi metrik terstruktur. Praktik terbaik adalah menanamkan timestamp pada dua titik, saat command diterima di service command, dan saat view model menyelesaikan update. Dengan data ini dapat dihitung latensi antar-langkah (persist latency, projection latency) dan latensi end-to-end. Penggunaan alat benchmarking seperti Artillery untuk mensintesis beban dan menyertakan probe query berkala yang meminta state terbaru sambil merekam timestamp, sehingga lag dapat diekstrak dari log klien dan log server yang ter-sinkronisasi. Eksperimen harus mencakup skenario steady, burst, dan recovery setelah kegagalan untuk menilai bagaimana eventual consistency lag berubah berdasarkan beban dan gangguan operasional (Cornelius & Jaswal, 2024).



Gambar 2.18. Contoh konfigurasi pengujian skrip Artillery

State rehydration time adalah waktu yang dibutuhkan untuk membangun kembali state operasional suatu agregat atau service dari event log, biasanya saat memulihkan instance baru (cold start) atau saat memutar ulang state untuk pemulihan. Pada sistem berbasis Event Sourcing, rehydration dapat dilakukan dengan memutar ulang semua event dari awal atau dari snapshot terakhir ditambah replay event setelah snapshot. Trade-off antara frekuensi snapshot dan waktu rehydration lebih sering memperpendek rehydration time tetapi menambah storage footprint dan overhead penyimpanan snapshot; sebaliknya, jarang snapshot mengurangi footprint tetapi memperpanjang waktu pemulihan. Untuk studi POS, pengujian rehydration time harus mengukur waktu mulai dari instans baru dinyalakan hingga service siap melayani query/command pada beban produksi, menggunakan skenario snapshot/no-snapshot dan variasi ukuran backlog event untuk menilai skenario worst-case dan typical-case.

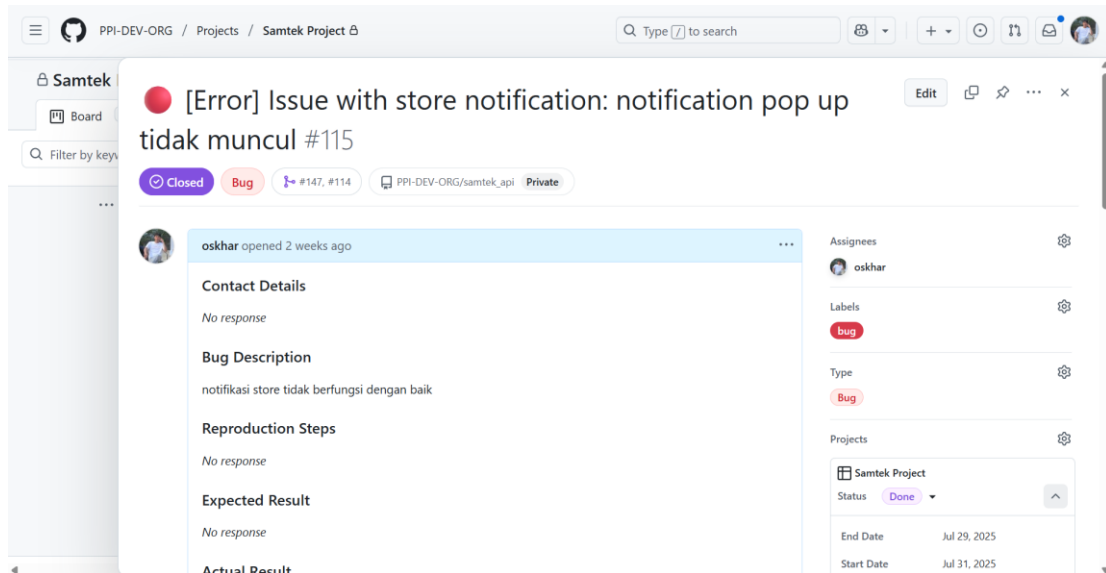
Storage footprint pada arsitektur event-driven mencakup ukuran event store, ukuran snapshot, dan metadata lain yang diperlukan (index, projections). Pertumbuhan event log bersifat akumulatif sehingga, tanpa kebijakan retensi atau compaction, storage akan tumbuh linier terhadap volume transaksi. Strategi manajemen storage harus mempertimbangkan biaya penyimpanan, I/O performance, dan efeknya pada operasi rehydration dan analisis historis. Pengukuran footprint yang tepat meliputi: bytes per event rata-rata, growth rate (GB per hari), overhead indeks, ukuran snapshot periodik, serta biaya operasi I/O (IOPS) saat menyimpan atau membaca event pada

beban puncak. Dalam kasus POS, pengukuran ini membantu menilai trade-off ekonomis antara mempertahankan history penuh untuk audit versus melakukan retensi tersegmentasi (misalnya archive older-than-N-days) yang mengurangi biaya tetapi mengorbankan kemudahan replay untuk periode lama.

Secara metodologis, hasil pengukuran ketiga dimensi harus dianalisis menggunakan metrik distribusi (median dan persentil tinggi), serta diuji signifikansinya menggunakan uji statistik antar-run untuk memastikan perbedaan bukan hasil kebisingan eksperimen. Kleppmann menekankan penting mengaitkan metrik teknis dan arsitektural dengan konsekuensi bisnis, contohnya menentukan ambang batas acceptable eventual consistency lag untuk tipe transaksi tertentu pada POS sehingga keputusan tentang snapshotting, retensi, dan arsitektur data dapat dibuat berdasarkan cost-benefit nyata. Laporan eksperimen akhir perlu menyajikan matriks trade-off yang memetakan perbedaan microservices+CQRS+ES vs monolith pada ketiga dimensi ini sehingga pembuat kebijakan arsitektural dapat memilih opsi yang paling sesuai dengan kebutuhan operasional dan anggaran (Kleppmann, 2017).

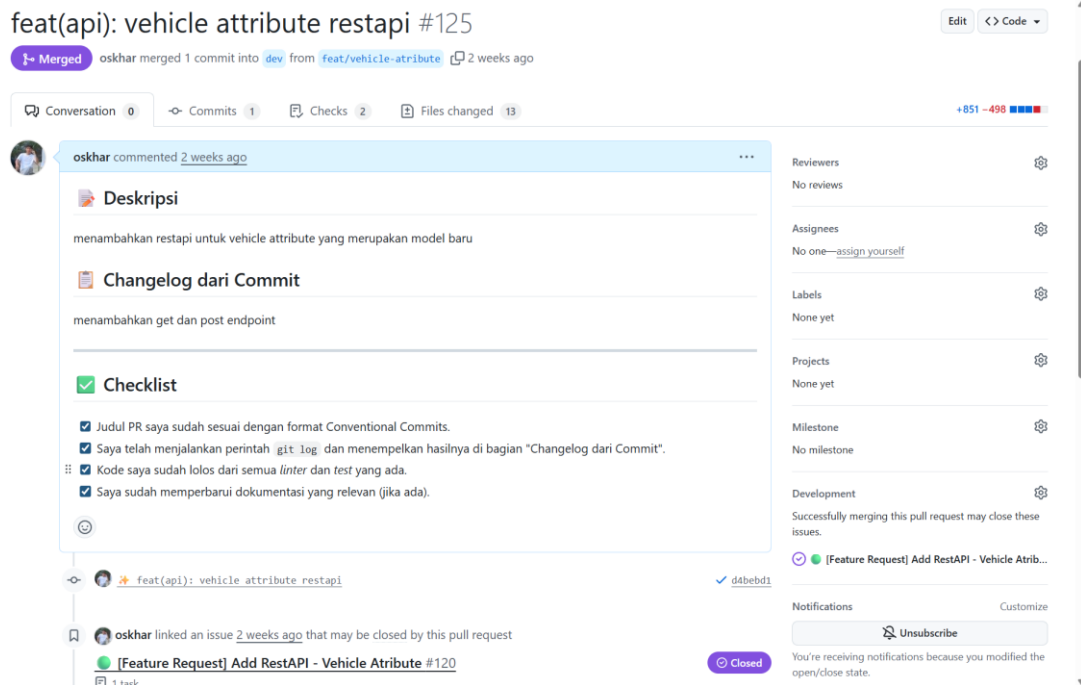
2.11 Pengujian Dimensi Developer

Pengukuran metrik developer harus dimulai dari kerangka metrik yang terdefinisi dan terkait langsung dengan unit arsitektural yang diuji; Taibi dan Systä menyarankan pendekatan berbasis dekomposisi dimana metrik developer dipetakan ke bounded context atau vertical slice yang sama dengan yang diuji pada dimensi teknis. Untuk lead time for changes, pendekatan praktis yang akan digunakan adalah mengukur waktu dari titik inisiasi perubahan (misalnya waktu pembuatan issue atau waktu commit pertama yang terkait dengan satu SCS smallest change set yang didefinisikan penelitian) sampai perubahan tersebut tersedia pada lingkungan staging/produksi (ditandai oleh merge ke branch release atau tag deploy). Data ini diambil dari timeline GitHub (timestamps issue assignment, first commit, PR merge, CI deploy) dan divalidasi dengan catatan harian developer yang mencatat waktu aktif pengerjaan tiap SCS; analisisnya menampilkan distribusi waktu (median, p95) dan perubahan lead time per iterasi eksperimen agar terlihat pengaruh arsitektur terhadap kecepatan pengiriman fitur. Pengukuran ini sesuai landasan Taibi & Systä yang menekankan keterkaitan eksplisit antara unit arsitektural dan metric developer.



Gambar 2.19. Contoh timeline GitHub untuk satu SCS

Lead time for changes memberi gambaran langsung tentang produktivitas dan alur nilai. Nogueira et al. menunjukkan dari survei praktisi bahwa pengukuran lead time berbasis artefak repositori (commit > merge > deploy) lebih objektif daripada survei subjektif dan sangat dihargai oleh tim industri sebagai indikator perubahan proses yang nyata. Dalam konteks penelitian POS, lead time yang lebih pendek pada arsitektur tertentu mencerminkan efektivitas decomposition by subdomain dan isolation tanggung jawab, karena perubahan kecil pada satu bounded context dapat dikembangkan dan dideploy tanpa menunggu seluruh sistem. Data GitHub menyediakan jejak temporal yang presisi untuk menghitung lead time, sedangkan developer logs membantu menjelaskan anomali (misalnya jeda karena dependency eksternal atau review panjang). Analisis berfokus pada seberapa sering lead time berada di luar ambang yang dapat diterima oleh operasi toko (misalnya > X jam) dan apakah pola ini konsisten antara monolith dan microservices+CQRS+ES.

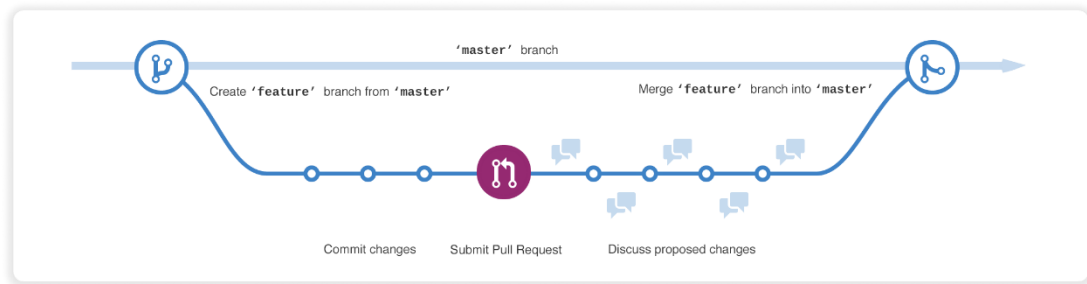


Gambar 2.20. Contoh catatan developer (developer log).

Context-switching count diukur sebagai frekuensi perpindahan fokus kerja developer antar tugas atau bounded context dalam satu unit waktu, dan ini diestimasi kuantitatif dari kombinasi dua sumber: GitHub (perubahan issue/PR yang dikerjakan dalam satu hari, jumlah branch aktif yang disentuh, interval waktu antar commit pada topik berbeda) dan log harian yang mencatat interupsi dan tugas yang diproses. Fragmentasi konteks (tingginya context-switching) berkorelasi negatif dengan throughput pengembangan dan kualitas komit; oleh karena itu pengukuran context-switching count harus menghitung perpindahan antar bounded context (misalnya berpindah dari Transaction ke Inventory) berdasarkan label issue atau struktur repo, ditambah verifikasi manual dari developer logs untuk mengidentifikasi interupsi non-teknis (rapat, penanganan insiden). Hasilnya dinyatakan sebagai rata-rata perpindahan per hari per developer dan distribusi durasi fokus (time-on-task) untuk menilai beban kognitif arsitektural.

Commit granularity dan churn adalah metrik kunci untuk memahami proses kerja dan kualitas pengembangan. Commit granularity diukur dengan metrik seperti jumlah commit per SCS, rata-rata baris kode yang diubah per commit, dan proporsi commit kecil (idealnya banyak commit kecil yang fokus) versus commit besar yang menggabungkan banyak perubahan. Churn diukur sebagai rasio baris yang

dihapus/dimodifikasi dibandingkan baris baru dalam rentang waktu tertentu dan termasuk metrik seperti jumlah revert, amend, atau force-push yang menunjukkan perubahan ulang pekerjaan. Pola commit yang baik mendukung refactoring dan evolusi API karena memudahkan review dan tracing perubahan; oleh karena itu penelitian ini akan mengekstraksi data commit dari GitHub untuk setiap SCS, mengkalkulasi commit-per-SCS, median ukuran commit (lines changed), churn rate selama tiap fase implementasi, serta membandingkan frekuensi revert antara arsitektur. Data developer logs akan membantu menafsirkan sebab churn tinggi (misalnya requirement berubah atau konflik integrasi). Analisis ini penting untuk mengukur apakah arsitektur microservices memperkecil commit granularity (lebih banyak commit kecil per SCS) dan menurunkan churn relatif terhadap monolith (Stocker & Zimmermann, 2023).



Gambar 2.21. Visualisasi histogram ukuran commit per SCS.

Dalam praktiknya, integrasi GitHub API dan proses parsing developer logs membutuhkan definisi yang konsisten untuk unit SCS dan aturan korelasi. Metodologi yang direkomendasikan adalah pertama membuat mapping issue → branch → PR untuk setiap SCS, kedua menanamkan tag atau konvensi pesan commit yang mengikat commit ke ID issue, ketiga mensinkronkan jam di mesin lokal developer dengan server untuk konsistensi timestamp; keempat menstandarkan format developer logs (tanggal, jam mulai/selesai, topik, interupsi) agar dapat diparse otomatis. Taibi dan Systä menekankan pentingnya repeatability pengukuran; oleh karena itu skrip ekstraksi GitHub, parser log, dan definisi metrik harus disertakan dalam artefak penelitian sehingga analisis lead time, context-switching, dan churn dapat direplikasi.

Akhirnya, relevansi metrik-metrik ini untuk studi POS sangat praktis, lead time yang panjang untuk perubahan kritis pada Transaction Service dapat memengaruhi waktu respons terhadap kebutuhan bisnis (misalnya penyesuaian harga atau fix bug pada checkout), high context-switching count menunjukkan arsitektur yang memaksa developer berpindah antar domain sehingga menurunkan produktivitas, dan commit churn tinggi menandakan ketidakstabilan requirement atau konflik integrasi yang dapat

menambah risiko regresi saat deploy. Metrik developer harus diinterpretasikan dalam konteks konsekuensi bisnis; penelitian ini akan mengaitkan metrik developer dengan outcome teknis (throughput, latency, error recovery) untuk menilai trade-off apakah peningkatan kinerja arsitektural sebanding dengan beban kerja dan fragmentasi tim yang muncul.

2.12 Studi Sejenis

Dalam penelitian ini, penulis mereferensi penelitian terdahulu untuk mematangkan pemahaman, berikut adalah tabel literatur review.

Tabel 2.1. Tabel Penelitian Sejenis

Judul	Deskripsi	Relevansi dengan Skripsi	Kelebihan	Kekurangan
Kleppmann (2017) <i>Designing Data-Intensive Applications</i>	Buku fundamental tentang prinsip desain aplikasi data-intensive (reliabilitas, skalabilitas, maintainabilitas, manajemen data terdistribusi).	Menjadi landasan teoretis utama untuk dimensi teknis & arsitektural.	Memberikan kerangka konseptual kuat untuk aplikasi data-intensive.	Tidak memberikan metodologi eksperimen; sifatnya teoretis, bukan evaluatif.
Lewis & Fowler (2014) <i>Microservices – A definition of this new architectural term</i>	Artikel seminal yang mendefinisikan microservices sebagai gaya arsitektur independen, scalable, dan loosely coupled.	Relevan sebagai dasar untuk perbandingan Monolith vs Microservices.	Memberikan definisi standar yang dipakai global.	Terlalu umum, tidak membahas performa atau dimensi developer.
Fowler (2005) <i>Event Sourcing</i>	Artikel awal yang memperkenalkan pola Event Sourcing untuk menyimpan state melalui event log.	Relevan untuk arsitektur POS hybrid yang menggabungkan CQRS + ES.	Konsep foundational bagi penelitian modern tentang event-driven system.	Tidak ada studi empiris, hanya definisi dan motivasi pattern.
Kabbedijk et al. (2012) <i>A case study of the</i>	Studi kasus implementasi CQRS di software bisnis	Memberi pembandingan empiris untuk	Analisis mendalam konsekuensi	Fokus ke CQRS saja, tidak menguji hybrid

<i>variability consequences of the CQRS pattern</i>	online, menyoroti konsekuensi variabilitas.	aspek CQRS di arsitektur Anda.	variabilitas pada skala industri.	pattern atau metrik multi-dimensi.
Fansha et al. (2021) <i>Load Test pada Microservice yang menerapkan CQRS dan Event Sourcing</i>	Studi empiris yang melakukan load test pada microservices dengan CQRS + Event Sourcing.	Sangat dekat dengan skripsi, terutama pada uji teknis performa.	Memberikan data uji throughput & latency untuk arsitektur serupa.	Tidak membandingkan dengan monolith, tidak mengukur dimensi developer.
Overeem et al. (2021) <i>An empirical characterization of event sourced systems and their schema evolution</i>	Studi empiris industri tentang evolusi skema pada sistem event sourcing.	Relevan dengan dimensi arsitektural (schema evolution, rehydration).	Memberikan insight praktis dari sistem industri nyata.	Fokus ke skema data, tidak menyentuh dimensi teknis maupun developer.
Ataei & Staegemann (2023) <i>Application of microservices patterns to big data systems</i>	Menerapkan pola microservices pada sistem big data.	Relevan untuk data-intensive system dengan microservices.	Menunjukkan adaptasi microservices pada skala data besar.	Tidak fokus ke POS atau evaluasi multi-dimensi.
Laigner et al. (2021) <i>Data management in microservices</i>	Survei praktik pengelolaan data dalam microservices (konsistensi, state rehydration, tantangan riset).	Relevan dengan dimensi arsitektural & data-intensive POS.	Menyediakan tinjauan luas tentang masalah data di microservices.	Bersifat survei, tidak ada eksperimen atau evaluasi performa.
Taibi & Systä (2020) <i>A Decomposition and Metric-Based Evaluation Framework for Microservices</i>	Framework evaluasi berbasis metrik untuk decomposition dan kualitas microservices.	Relevan dengan metodologi pengukuran skripsi Anda.	Memberikan metrik formal untuk evaluasi arsitektur.	Tidak fokus ke hybrid arsitektur, tidak mencakup dimensi developer detail.

Christian & Bisma (2021) <i>Studi Perbandingan Performa Aplikasi Web Monolitik dan Microservice Berbasis Apache Kafka</i>	Studi empiris membandingkan performa monolith vs microservices berbasis Kafka.	Relevan untuk baseline perbandingan Monolith vs Microservices.	Memberikan hasil konkret perbandingan performa dua arsitektur.	Tidak membahas CQRS/ES, tidak mengukur dimensi developer & arsitektural kompleks.
---	--	--	--	---

Tabel 2.2. Tabel Perbandingan

No	Judul	Perbandingan Hybrid	Vertical Slice Implementation	C4 Model	Sistem Data-Intensive	Pengujian & Pengukuran		
						Dimensi Developer	Dimensi Teknis	Dimensi Arsitektural
1	Oskhar et al. <i>Evaluasi Multi-Dimensi Arsitektur Pattern Modern Backend pada Aplikasi Data-Intensif</i>	V	V	V	V	V	V	V
2	Kleppmann (2017) <i>Designing Data-Intensive Applications</i>	X	X	X	V	X	V	V
3	Lewis & Fowler (2014) <i>Microservices - A definition of this new architectural term</i>	X	X	X	V	X	X	V
4	Fowler (2005) <i>Event Sourcing</i>	X	X	X	V	X	X	V
5	Kabbedijk et al. (2012) <i>A case study of the variability consequences of the CQRS pattern in online business Xsoftware</i>	X	X	X	V	X	V	V
6	Fansha et al. (2021) <i>Load Test pada Microservice yang menerapkan CQRS dan Event Sourcing</i>	V	X	X	V	X	V	V
7	Overeem et al. (2021) <i>An empirical characterization of event sourced systems and their schema evolution — Lessons from industry</i>	X	X	X	V	X	X	V
8	Ataei & Staegemann (2023) <i>Application of microservices patterns to big data systems</i>	X	X	X	V	X	V	V
9	Laigner et al. (2021) <i>Data management in microservices: State of</i>	X	X	X	V	X	X	V

	<i>the practice, challenges, and research directions</i>							
10	Taibi & Systä (2020) <i>A Decomposition and Metric-Based Evaluation Framework for Microservices</i>	X	X	X	V	V	V	V
11	Christian & Bisma (2021) <i>Studi Perbandingan Performa Aplikasi Web Monolitik dan Microservice Berbasis Apache Kafka</i>	V	X	X	V	X	V	X

Berikut ringkasan hasil dari tinjauan pustaka (Studi Sejenis) berdasarkan Tabel 2.1 dan Tabel 2.2 yang menunjukkan temuan utama dan implikasinya terhadap penelitian ini:

1. Literatur mendukung kombinasi Microservices + CQRS + Event Sourcing sebagai solusi yang kuat untuk aplikasi data-intensive, terutama dalam hal skalabilitas dan auditabilitas, tetapi banyak kajian menekankan adanya trade-off operasional.
2. Studi empiris load-testing (Fansha et al., 2021) menunjukkan potensi peningkatan throughput/ketahanan pada arsitektur hybrid, namun seringkali belum dibandingkan langsung dengan baseline monolith pada metrik yang sama.
3. Event Sourcing memberikan keuntungan audit dan fleksibilitas schema (upcasting, replay), tetapi secara konsisten dicatat meningkatkan storage footprint dan memperpanjang state rehydration time bila snapshotting tidak dioptimalkan.
4. CQRS memperjelas pemisahan jalur baca/tulis sehingga optimasi query lebih mudah, namun menimbulkan eventual-consistency lag yang perlu diukur dan dikelola dalam konteks POS.
5. Banyak kajian menekankan perlunya kerangka evaluasi berbasis metrik (decomposition + metrics) untuk memilih boundary layanan; ini mendukung pendekatan vertical-slice dan pengukuran metrik developer yang digunakan pada penelitian ini.
6. Celah penting: sedikit penelitian yang melakukan evaluasi multi-dimensi sekaligus (teknis, arsitektural, developer) dan membandingkannya dengan monolith sebagai baseline itulah kontribusi utama yang ditargetkan skripsi ini

BAB III

METODOLOGI PENELITIAN

3.1 Objek Penelitian

Objek penelitian ini adalah *backend* aplikasi Point of Sale (POS) yang bersifat *data-intensive*. Sistem POS dipilih karena secara alami menghadapi volume transaksi tinggi, kebutuhan akses data cepat, serta integrasi lintas modul (inventori, transaksi, laporan, pembayaran). Karakteristik ini menjadikannya konteks ideal untuk mengevaluasi keputusan arsitektural modern.

Fokus penelitian bukan sekadar membandingkan arsitektur monolith dengan kombinasi Microservices + CQRS + Event Sourcing, melainkan menginvestigasi trade-off keputusan arsitektural yang muncul. Investigasi ini menyoroti bagaimana pilihan desain memengaruhi tiga dimensi utama:

1. Dimensi Teknis & Performa

Mengukur throughput, latensi (p50, p95, p99), *success rate*, serta konsumsi sumber daya.

2. Dimensi Arsitektural

Menganalisis *eventual consistency lag*, *state rehydration time*, dan *storage footprint* sebagai konsekuensi langsung dari pola desain.

3. Dimensi Developer

Mengkaji dampak arsitektur pada pengalaman pengembangan melalui metrik *lead time for changes*, *commit granularity & churn*, serta *context-switching count*.

Monolith digunakan hanya sebagai baseline untuk memastikan hasil evaluasi lebih obyektif (pengujian monolith dengan lingkungan pengembangan dan lingkungan uji yang sama), bukan untuk menentukan mana yang lebih baik atau lebih buruk. Dengan baseline ini, penelitian dapat menguji apakah arsitektur modern benar-benar memberikan nilai tambah yang relevan untuk jangka panjang, atau justru memperkenalkan kompleksitas baru.

3.2 Tahap Implementasi

Tahap implementasi penelitian ini menggunakan pendekatan Rapid Application Development (RAD) yang disesuaikan dengan konteks penelitian, yaitu investigasi keputusan arsitektural melalui pendekatan *vertical slice*. Implementasi dibagi ke dalam sembilan tahap utama.

3.2.1 Spesifikasi Kebutuhan (Vertical Slice)

Tahap awal adalah menentukan kebutuhan sistem yang difokuskan pada satu *vertical slice* yang merepresentasikan alur end-to-end dari sistem Point of Sale (POS). Vertical slice ini dipilih agar dapat menguji aspek penting arsitektural dalam skala kecil namun utuh, misalnya proses transaksi yang menghasilkan pembaruan data inventori hingga tersedianya laporan ringkas. Hasil dari tahap ini berupa definisi *user story* yang singkat namun jelas, kontrak API dalam format OpenAPI, serta diagram arsitektur tingkat container (C4 container diagram) yang menggambarkan alur proses pada slice tersebut. Semua kebutuhan dirumuskan secara terukur agar dapat langsung digunakan sebagai acuan pada tahap implementasi berikutnya.

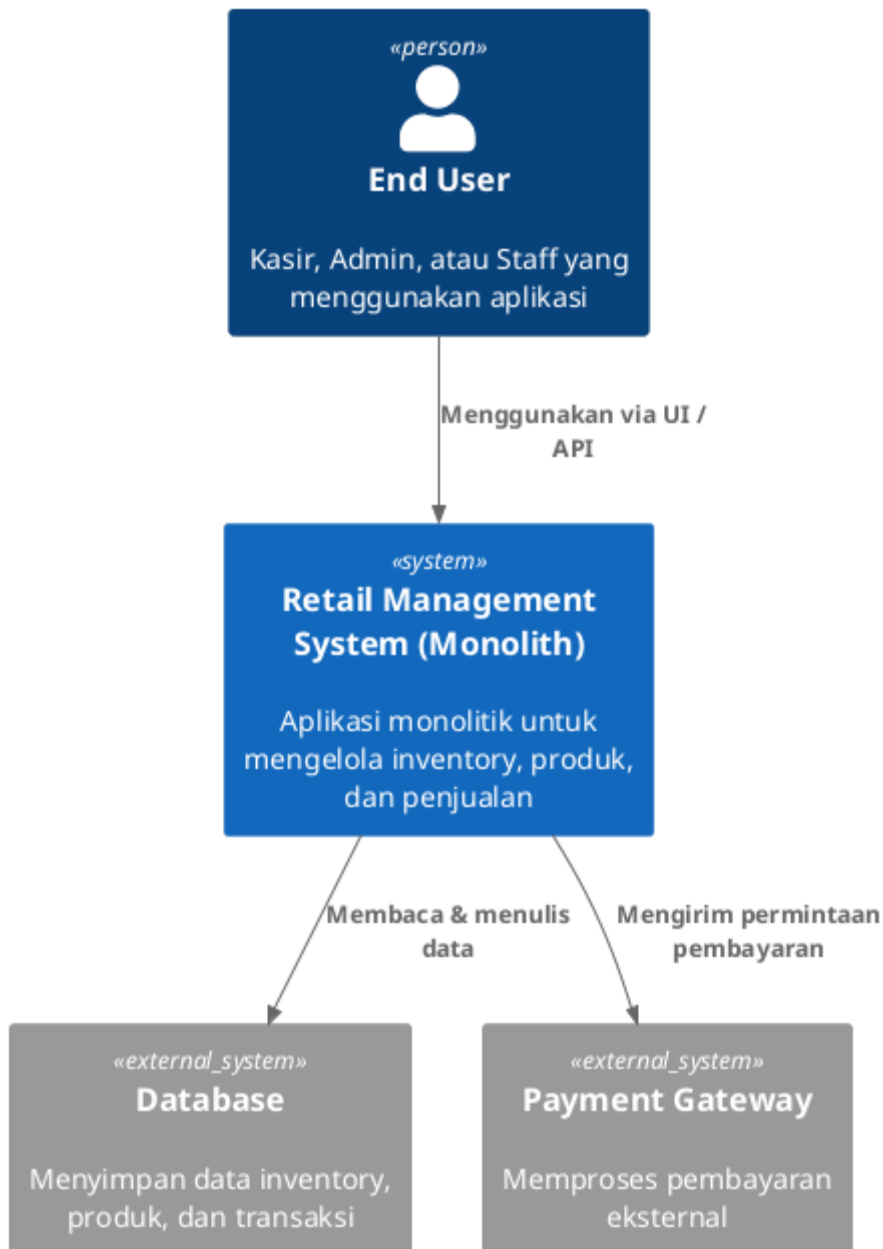
3.2.2 Standarisasi Repositori (Source Code Standardization - SCS)

Tahap berikutnya adalah menyiapkan repositori yang terstandarisasi di GitHub. Struktur repositori disusun agar mudah diatur baik dalam bentuk *mono-repo* maupun *multi-repo*, dengan penetapan konvensi commit, branch, dan tag untuk setiap *smallest change set* (SCS). Standarisasi ini mencakup penulisan file README, CONTRIBUTING, serta template untuk issue dan pull request. Selain itu, otomatisasi dasar seperti GitHub Actions untuk linting dan pengujian juga diterapkan sejak awal. Tujuannya adalah menciptakan landasan pengembangan yang konsisten sekaligus memudahkan pengambilan data metrik developer, karena setiap perubahan yang tercatat di repositori akan menjadi bahan analisis pada tahap evaluasi.

3.2.3 Implementasi Kode (Monolith ke Hybrid Vertical Slice)

Pada tahap implementasi kode, pembangunan dimulai dari arsitektur monolith untuk vertical slice yang sudah ditentukan. Framework yang digunakan adalah NestJS (Node.js) karena mendukung pengembangan modular dan cepat. Setelah slice dasar dalam bentuk monolith berfungsi, dilakukan refactoring bertahap ke arah arsitektur hybrid dengan mengekstraksi layanan tertentu menjadi microservice kecil, misalnya layanan transaksi dan layanan inventori. Pendekatan *extract-as-needed* ini membuat implementasi lebih terukur dan dapat dikerjakan secara realistis oleh satu orang.

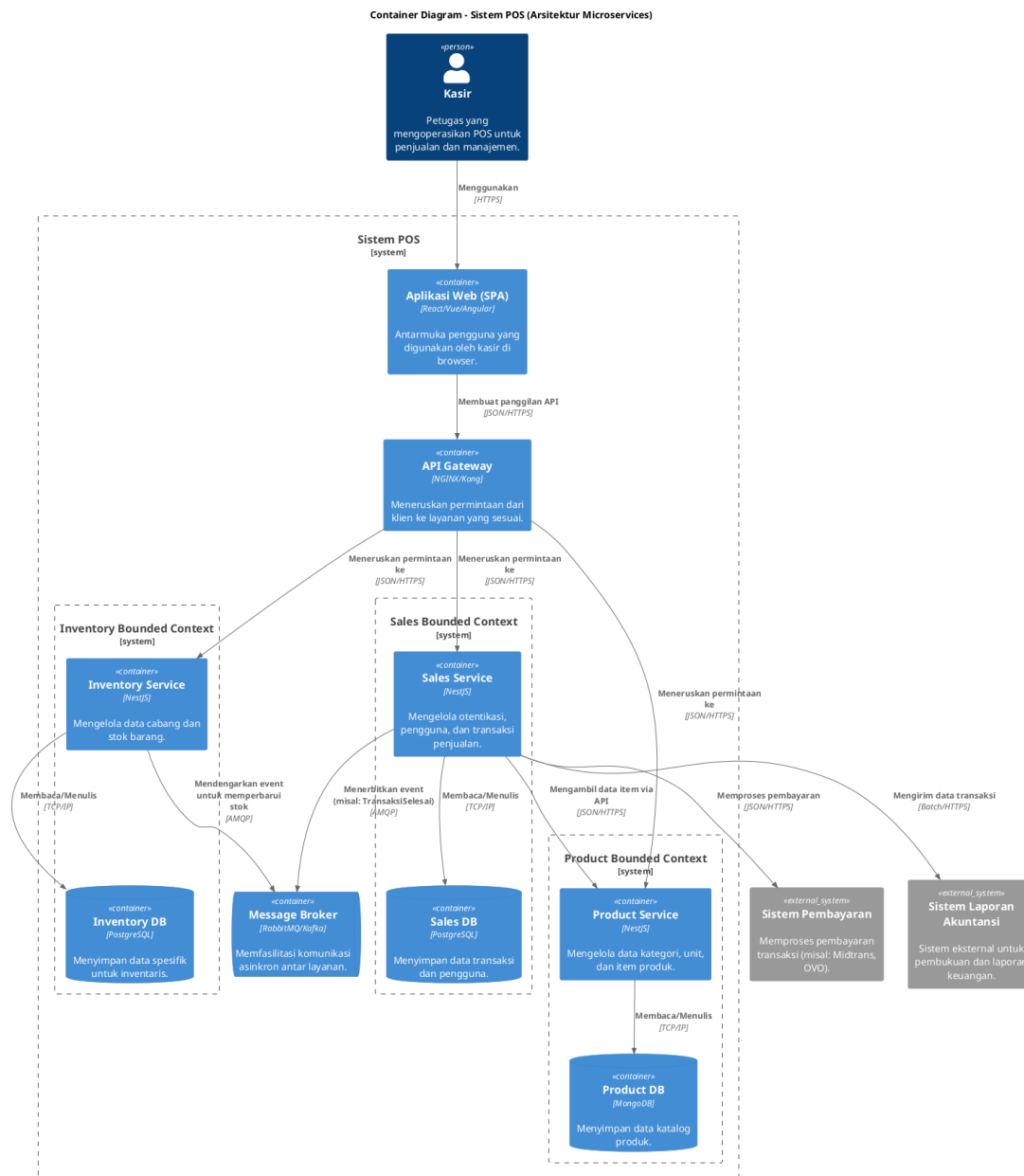
Lingkungan pengembangan diatur menggunakan Docker Compose sehingga layanan monolith maupun microservice dapat dijalankan secara konsisten di mesin lokal.



Gambar 3.1. Desain Context C4 Model Arsitektur Monolith

Pada arsitektur monolith, seluruh domain aplikasi seperti inventory, product, dan sales (termasuk auth, transaction, dan user) diimplementasikan dalam satu sistem terpadu yang menangani seluruh proses bisnis mulai dari pengelolaan produk,

manajemen stok, hingga transaksi penjualan. Sistem ini menjadi satu kesatuan yang berinteraksi langsung dengan aktor eksternal, yaitu kasir sebagai pengguna utama, admin sebagai pengelola data dan laporan, serta sistem eksternal seperti payment gateway dan accounting system. Desain ini sederhana karena hanya memiliki satu titik integrasi, namun seluruh modul saling terikat erat sehingga setiap perubahan atau penambahan fitur akan mempengaruhi keseluruhan aplikasi, termasuk dari sisi skalabilitas dan fleksibilitas pengembangan.



Gambar 3.2. Desain Context C4 Model Arsitektur Monolith

Pada arsitektur microservices, sistem POS dipecah menjadi beberapa layanan terpisah yang masing-masing bertanggung jawab atas satu bounded context, seperti *Transaction Service*, *Inventory Service*, *Product Service*, dan *Auth Service*, yang seluruhnya diakses melalui *API Gateway*. Pola *CQRS* diterapkan dengan memisahkan jalur *command* (penulisan data) dari *query* (pembacaan data), sementara *Event Sourcing* digunakan untuk menyimpan setiap perubahan dalam bentuk event pada *Event Store* yang kemudian didistribusikan melalui *Event Bus* untuk menjaga konsistensi antar layanan. Pendekatan ini memungkinkan layanan berkomunikasi secara *event-driven* dan memperbarui *read models* untuk kebutuhan laporan serta integrasi ke sistem eksternal seperti *payment gateway* dan *accounting system*. Arsitektur ini memberikan fleksibilitas dan skalabilitas yang lebih tinggi dibanding monolith, namun juga membawa kompleksitas tambahan dalam hal orkestrasi, sinkronisasi data, dan pengelolaan *eventual consistency*.

3.2.4 Penerapan CQRS dan Event Sourcing

Untuk menambahkan dimensi arsitektural, dilakukan implementasi terbatas dari pola CQRS dan Event Sourcing. Model penulisan (write model) menghasilkan event immutable yang disimpan dalam event store sederhana, sedangkan model pembacaan (read model) terproyeksi dari event log menjadi data yang sudah terdenormalisasi. Mekanisme snapshot juga diterapkan secara sederhana untuk mengurangi waktu rehidrasi state ketika sistem diinisialisasi ulang. Setiap event yang tercatat juga dihitung ukuran dan jumlahnya, yang nantinya menjadi data penting untuk mengukur *storage footprint*.

3.2.5 Instrumentasi dan Pencatatan Metrik Developer

Agar implementasi dapat sekaligus menjadi sumber data penelitian, dilakukan instrumentasi untuk mengukur metrik developer. Otomatisasi dilakukan dengan memanfaatkan API GitHub untuk mengekstrak timestamp dari issue, commit, pull request, hingga merge, yang kemudian dihitung menjadi lead time per *smallest change set*. Selain itu, pencatatan manual juga dilakukan oleh peneliti dalam bentuk catatan harian singkat mengenai waktu mulai, waktu selesai, interupsi, serta topik pekerjaan. Data ini dikompilasi menjadi format CSV sederhana yang mudah dianalisis. Dari repositori, diambil pula data commit size, churn, dan frekuensi perubahan untuk melengkapi analisis kuantitatif.

3.2.6 Unit Testing dan Integration Testing

Kualitas kode dijaga melalui pengujian otomatis sejak tahap awal. Framework Jest digunakan untuk melakukan unit test pada setiap service dan logic, sedangkan integration test dilakukan dengan menggunakan *supertest* atau skenario end-to-end sederhana yang memanggil API pada vertical slice. Data uji dibuat kecil namun representatif agar setiap fungsi utama dapat tervalidasi. Seluruh pengujian ini dijalankan otomatis pada GitHub Actions sebelum proses merge sehingga konsistensi kode selalu terjaga.

3.2.7 Setup Pengujian Beban dan Observabilitas

Tahap ini berfokus pada pengujian performa arsitektur. Profil beban disimulasikan menggunakan Artillery dengan pola uji warm-up, steady-state, spike, dan cool-down. Dari pengujian ini diperoleh metrik throughput, latency (p50, p95, p99), serta success rate. Selain itu, metrik penggunaan sumber daya (CPU, memori) juga dikumpulkan. Untuk mengukur sifat arsitektural yang lebih mendalam, ditambahkan pengukuran *eventual consistency lag* melalui pencatatan timestamp event, serta *state rehydration time* ketika sistem melakukan pemulihan state dari event log.

3.2.8 Continuous Development Cycle

Seluruh implementasi dijalankan dengan prinsip agile yang iteratif. Setiap iterasi difokuskan pada penyelesaian satu hingga dua *smallest change set* yang menghasilkan kode fungsional, teruji, dan tercatat metriknya. Backlog sederhana dikelola menggunakan GitHub Issues, dan setiap iterasi menghasilkan artefak yang terdokumentasi seperti skrip Artillery, konfigurasi environment, serta dataset sintetis. Dengan pendekatan ini, setiap siklus pengembangan tidak hanya menambah fitur, tetapi juga menambah data eksperimen yang dapat dianalisis pada tahap berikutnya.

3.2.9 Deliverable dan Artefak Penelitian

Hasil dari seluruh tahap implementasi adalah kumpulan artefak yang terstruktur dan siap digunakan untuk pengujian serta analisis pada Bab IV. Artefak tersebut mencakup kode sumber di repositori, diagram arsitektur (C4), kontrak API (OpenAPI), skrip load test, konfigurasi Docker Compose atau Kubernetes minimal, dataset sintetis, serta skrip ekstraksi metrik developer. Semua artefak ini tidak hanya menjadi produk teknis, tetapi juga menjadi bukti konkret untuk mendukung investigasi akademik terhadap keputusan arsitektural yang telah diambil.

3.3 Tahap Penelitian

Tahapan penelitian adalah jantung dari seluruh penelitian ini, bagaimana jantung penelitian ini diteliti dengan eksperiment terukur dan valid, berikut adalah seluruh tahapan dari proses penelitian ini.

3.3.1 Perancangan eksperimen & definisi variable

Rancang eksperimen menurut tujuan penelitian, variabel bebas = arsitektur (Monolith vs Microservices+CQRS+ES), variabel terikat = metrik teknis (throughput, p50/p95/p99, success rate, CPU/mem), metrik arsitektural (eventual consistency lag, state rehydration time, storage footprint), dan metrik developer (lead time for changes, commit granularity, churn, context-switching). Tetapkan vertical slice yang identik fungsional untuk kedua arsitektur (mis. alur transaksi → inventori → laporan) agar perbandingan adil. Hasilnya adalah daftar skenario uji, matriks metrik, dan mapping endpoint / titik pengukuran pada diagram C4.

3.3.2 Persiapan lingkungan eksperimen & artefak reproduktif

Siapkan lingkungan minimal yang dapat dijalankan sendiri (Docker Compose untuk lokal; opsi kecil Kubernetes bila perlu). Buat repositori terstandar (README, CONTRIBUTING, template PR/issue, konvensi commit untuk SCS) dan pipeline CI ringan (GitHub Actions) untuk lint/test otomatis. Buat kontrak API (OpenAPI) dan C4 container diagram untuk tiap vertical slice sehingga semua titik pengukuran terdokumentasi. Artefak: repo, Docker Compose / k8s manifests minimal, OpenAPI, diagram C4.

3.3.3 Pembuatan dataset sintetis & profil beban

Bangun generator dataset sintetis yang merepresentasikan karakter POS: katalog produk (skala kecil ke sedang, contoh 1k–10k item), daftar pelanggan, pola transaksi (steady, spike, mixed read/write). Definisikan workload profile Artillery: warm-up, steady-state, spike (burst), cool-down. Pastikan seed deterministic agar run dapat diulang. Artefak: skrip generator data, file dataset (.csv/.json), profil Artillery. (Detail profil uji dan skrip disimpan dalam repo).

3.3.4 Implementasi vertical slice Monolith lalu Hybrid

Implementasikan vertical slice dahulu sebagai monolith (NestJS/Node.js) lalu lakukan refactoring bertahap menjadi hybrid: ekstraksi layanan Transaction dan Inventory menjadi microservices, terapan CQRS (write model + read model), dan

Event Sourcing minimal (event store sederhana + mekanisme snapshot). Pendekatan bertahap ini membuat pekerjaan terukur dan realistis untuk satu peneliti. Pastikan kedua implementasi tetap memberikan API fungsional yang sama untuk pengujian. Artefak: kode monolith, kode microservices, skrip Docker Compose untuk tiap mode.

3.3.5 Instrumentasi observabilitas & pencatatan metrik

Tambahkan instrumentasi: endpoint metrik di setiap service (mis. /metrics atau endpoint JSON), log terstruktur dengan timestamp untuk setiap command/event/projection, dan pengumpulan resource metrics (docker stats / cAdvisor + Prometheus jika memungkinkan). Untuk pengukuran eventual consistency, tanam timestamp pada saat command diterima dan pada saat projection selesai. Untuk developer metrics, siapkan skrip ekstraksi GitHub API dan format CSV untuk developer log manual (waktu mulai/selesai, interupsi, topik). Artefak: konfigurasi monitoring, skrip ekstraksi metrik, template developer log.

3.3.6 Pengujian teknis (load testing) dengan prosedur rinci

Jalankan skenario Artillery yang telah didefinisikan untuk setiap arsitektur dengan urutan: warm-up → steady-state → spike → cool-down. Untuk setiap skenario lakukan beberapa run independen (minimal N run — catat tiap run) agar dapat menghitung distribusi metrik. Ambil metrik end-to-end dari klien (p50/p95/p99, throughput, success rate) dan metrik server (CPU, mem, I/O). Pisahkan metrik klien & server untuk analisis bottleneck. Simpan semua hasil run sebagai artefak (JSON/CSV) untuk analisis.

3.3.7 Pengujian dimensi arsitektural

Eventual consistency lag yang jalankan beban tulis dan sesekali lakukan probe query yang merekam timestamp; hitung selisih command→projection. Sertakan scenario steady, spike, dan recovery setelah failure.

State rehydration time merupakan ukur waktu bootstrap service baru sampai siap melayani pada kondisi backlog berbeda (contoh: 1k, 10k, 100k event) menggunakan dua strategi: replay full event log vs snapshot + replay delta. Catat time-to-ready dan resource yang dipakai.

Storage footprint dengan hitung bytes per event rata-rata, ukuran snapshot, growth rate (GB/day) selama run uji. Dokumentasikan trade-off snapshot frequency vs rehydration time. Artefak: skrip pengukuran lag/rehydration, tabel footprint.

3.3.8 Failure injection & uji ketahanan operasional

Lakukan failure-injection terkontrol: hentikan container layanan secara acak, tambahkan latency paket (tc/netem), kurangi I/O/CPU (stress-ng atau Docker resource limits), hingga mensimulasikan throttling event queue. Ukur RTO (waktu pulih), tingkat kehilangan transaksi (jika ada), dan efektivitas replay. Dokumentasikan langkah recovery (snapshot reload, replay) agar dapat mereplikasikan hasil.

3.3.9 Pengukuran metrik developer (praktik & ekstraksi)

Definisikan SCS (Smallest Change Set) secara tegas dengan mapping issue→branch→PR→merge. Gunakan GitHub API untuk mengekstrak timestamps (first commit → PR open → merge/deploy) untuk menghitung lead time per SCS. Ekstrak ukuran commit (lines added/removed), jumlah commit per SCS (commit granularity), churn (rasio modifikasi/hapus terhadap baris baru), dan jumlah revert/amend. Hitung context-switching dari perubahan issue/branch yang berbeda dalam satu hari ditambah catatan manual developer log untuk memverifikasi interupsi. Artefak: skrip ekstraksi GitHub, dataset developer_metrics.csv, contoh developer log.

3.3.10 Pengumpulan, validasi & penjagaan kualitas data

Standarkan format log dan timestamp (sinkronisasi jam mesin), buat pipeline ekspor ke CSV/JSON, validasi integritas (cek missing values, alignment timestamps antara client/server), dan simpan metadata eksperimen (versi Node.js/NestJS, library, konfigurasi DB, ukuran snapshot, parameter Artillery). Semua run harus memiliki metadata lengkap supaya analisis reproducible. Artefak: folder eksperimen dengan naming convention + README eksperimen.

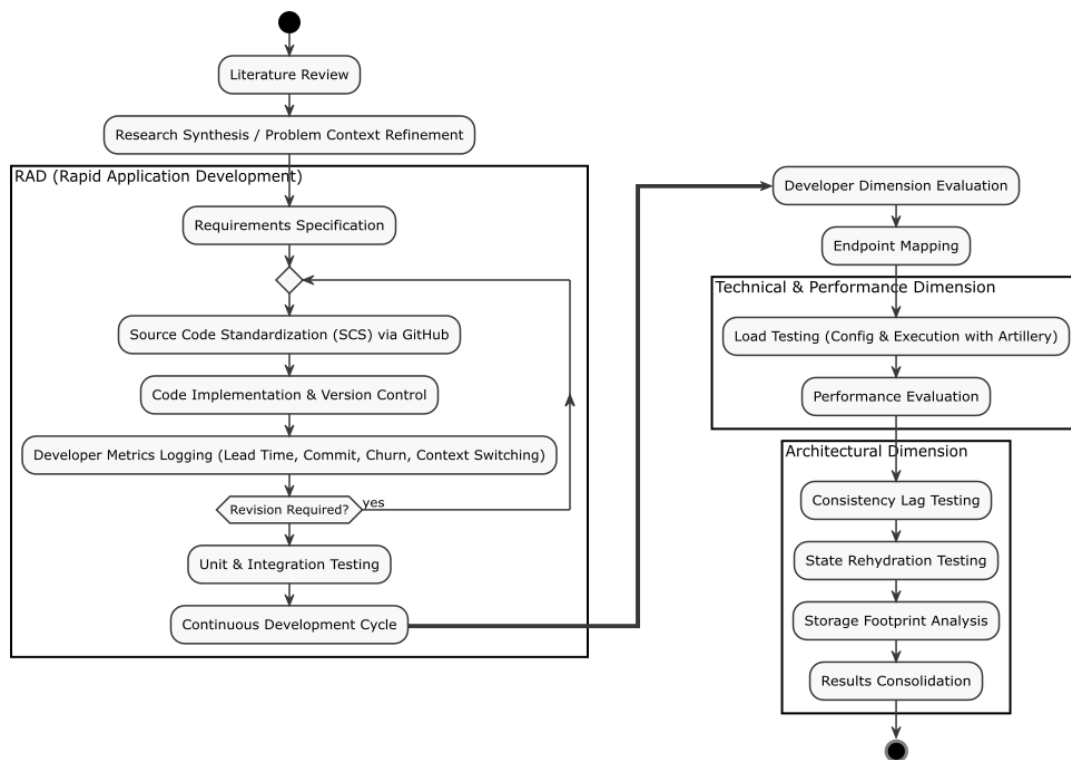
3.3.11 Analisis data & uji statistic

Untuk tiap metrik lakukan analisis deskriptif (median, p50/p95/p99, IQR), tambahkan interval kepercayaan bila memungkinkan. Untuk membandingkan arsitektur gunakan uji statistik yang sesuai: cek normalitas → gunakan t-test bila asumsi normal terpenuhi, atau Mann–Whitney U (non-parametric) bila tidak; laporkan ukuran efek. Gabungkan analisis kuantitatif dengan matriks trade-off (mis. throughput vs storage footprint vs lead time developer) untuk menjawab pertanyaan penelitian. Visualisasikan hasil (grafik persentil latency, growth storage, distribusi lead time) dan sertakan interpretasi yang mengaitkan hasil ke konsekuensi praktis.

3.3.12 Dokumentasi, artefak akhir & reproducibility penelitian

Penyusunan semua artefak akhir berikut kode sumber (branch/commit tag untuk tiap eksperimen), OpenAPI, C4 diagrams, dataset sintetik, skrip Artillery, konfigurasi Docker/k8s, skrip ekstraksi metrik developer, dan notebook analisis/CSV output. Sertakan petunjuk reproduksi langkah-per-langkah di README eksperimen dan package artefak ke dalam satu folder/zip yang bisa diserahkan bersama laporan. Artefak ini merupakan deliverable utama untuk Bab IV–V.

3.4 Kerangka Penelitian



Gambar 3.3. Diagram Alir Kerangka Penelitian

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menguraikan hasil implementasi, data pengujian, dan analisis mendalam mengenai evaluasi multi-dimensi antara arsitektur Monolith (Baseline) dan arsitektur Hybrid (Microservices + CQRS + Event Sourcing). Pembahasan difokuskan pada pembuktian empiris mengenai trade-off antara performa teknis dengan kompleksitas arsitektural serta beban kerja pengembang.

4.1 Gambaran Umum Implementasi

Bab ini menjelaskan bentuk konkret implementasi sistem yang digunakan sebagai objek eksperimen, dengan fokus pada satu vertical slice kritis dari aplikasi Point of Sale (POS), yaitu alur “Pembuatan Transaksi Penjualan hingga Pemotongan Stok Inventori”. Vertical slice tersebut dipilih karena merepresentasikan alur bisnis end-to-end yang lazim pada sistem POS dan melibatkan interaksi lintas modul inti seperti transaksi, produk, dan inventori. Dengan demikian, peneliti dapat mengevaluasi konsekuensi arsitektural pada konteks yang cukup realistis tanpa harus membangun seluruh fitur sistem secara menyeluruh.

Secara fungsional, vertical slice yang diimplementasikan mencakup proses ketika kasir mengirimkan data keranjang belanja berisi daftar item dan kuantitas, sistem memeriksa ketersediaan stok untuk setiap item, kemudian mencatat transaksi penjualan yang sah dan memperbarui stok inventori sesuai barang yang terjual. Setelah transaksi tersimpan, sistem juga menyediakan data transaksi beserta informasi stok terkini melalui endpoint pembacaan yang dapat dimanfaatkan untuk kebutuhan monitoring dan pelaporan sederhana. Kedua varian arsitektur, baik monolith maupun hybrid, dirancang untuk memenuhi ruang lingkup fungsional yang sama agar perbedaan hasil pengujian yang diperoleh dapat ditelusuri langsung ke perbedaan pola arsitektur backend, bukan ke perbedaan perilaku bisnis.

Pada arsitektur monolith yang berperan sebagai baseline, seluruh logika domain yang berkaitan dengan transaksi, produk, inventori, autentikasi, dan pengguna terhimpun dalam satu aplikasi backend berbasis Node.js dan NestJS. Modul-modul tersebut hidup dalam satu codebase dan dijalankan sebagai satu proses aplikasi yang berkomunikasi melalui pemanggilan fungsi di dalam proses yang sama. Data transaksi,

produk, dan inventori disimpan pada satu basis data relasional PostgreSQL yang menjalankan peran sebagai pusat penyimpanan tunggal. Operasi bisnis yang bersifat kritis, seperti pembuatan transaksi penjualan dan pemotongan stok, dibungkus dalam transaksi basis data yang bersifat ACID sehingga konsistensi data dijaga secara kuat pada satu boundary sistem. Dalam kerangka model C4, pendekatan ini dipresentasikan sebagai satu container aplikasi POS yang berinteraksi dengan aktor eksternal seperti kasir, admin, serta sistem eksternal semisal payment gateway dan sistem akuntansi.

Sebaliknya, arsitektur hybrid memecah tanggung jawab tersebut ke dalam beberapa layanan terpisah dengan mengadopsi kombinasi pola microservices, CQRS, dan Event Sourcing. Vertical slice yang sama diimplementasikan melalui serangkaian layanan yang saling berkoordinasi. Transaction Service menangani penerimaan perintah pembuatan transaksi dari klien, melakukan validasi awal, dan menghasilkan event yang merepresentasikan terjadinya transaksi penjualan. Inventory Service mengonsumsi event tersebut untuk melakukan pemotongan stok dan pada gilirannya menghasilkan event perubahan stok yang relevan dengan domain inventori. Di sisi lain, Query Service bertugas membangun dan memelihara read model terdenormalisasi dengan mengonsumsi rangkaian event dari Event Store, sehingga klien dapat mengakses data transaksi dan stok melalui jalur baca yang dioptimalkan sesuai prinsip CQRS.

Penyimpanan perubahan state pada arsitektur hybrid tidak lagi bergantung pada satu tabel state terkini, melainkan pada Event Store berbasis PostgreSQL yang menyimpan setiap perubahan sebagai event append-only yang bersifat immutable. Mekanisme ini memungkinkan rekonstruksi state melalui replay event dan didukung oleh penerapan snapshot sederhana untuk mengurangi waktu rehidrasi saat layanan diinisialisasi ulang. Distribusi event antar layanan dilakukan melalui message broker RabbitMQ yang mengimplementasikan pola komunikasi event-driven. Dengan pemisahan jalur tulis (command) dan jalur baca (query) serta penggunaan event sebagai sumber kebenaran, arsitektur ini secara teoritis menawarkan skalabilitas dan kemampuan audit yang lebih baik, namun sekaligus memperkenalkan kompleksitas tambahan seperti potensi keterlambatan propagasi data dan kebutuhan orkestrasi yang lebih rumit.

Kedua arsitektur tersebut dijalankan pada lingkungan eksperimen yang dikendalikan. Backend ditempatkan pada sebuah mesin virtual dengan konfigurasi 4 vCPU, 8 GB RAM dan media penyimpanan SSD. Seluruh komponen, baik aplikasi monolith maupun himpunan layanan pada arsitektur hybrid beserta basis data dan

message broker, didefinisikan dan dijalankan menggunakan Docker Compose. Konfigurasi ini dipilih agar peneliti dapat mengatur batasan sumber daya per container dan memantau konsumsi CPU maupun memori secara konsisten. Generator beban Artillery.io dijalankan pada node terpisah di jaringan yang sama, sehingga beban uji tidak mengganggu kapasitas komputasi backend dan pengaruh latensi jaringan eksternal dapat diminimalkan. Tumpukan teknologi inti yang digunakan pada kedua arsitektur dijaga konsisten, yaitu Node.js dan NestJS untuk backend serta PostgreSQL sebagai basis data utama, dengan tambahan RabbitMQ sebagai message broker khusus untuk varian hybrid.

Sebelum digunakan dalam pengujian performa dan pengukuran metrik arsitektural, kedua varian arsitektur melalui tahapan validasi fungsional melalui kombinasi unit testing dan integration testing. Pengujian dilakukan terhadap rangkaian skenario yang sama, di antaranya mengirimkan payload transaksi yang valid, memverifikasi bahwa transaksi ditolak ketika stok tidak mencukupi, memastikan transaksi sah tercatat dengan benar ketika stok tersedia, serta memeriksa bahwa stok inventori berkurang sesuai kuantitas item yang terjual dan dapat diakses kembali melalui endpoint pembacaan. Pada arsitektur monolith, seluruh rangkaian ini dieksekusi dalam satu proses aplikasi terintegrasi, sedangkan pada arsitektur hybrid rangkaian yang sama tercapai melalui aliran event yang melibatkan Transaction Service, Inventory Service, Event Store, dan Query Service. Hasil pengujian menunjukkan bahwa dari sudut pandang klien, kedua arsitektur memberikan perilaku fungsional yang ekuivalen. Kondisi ini menjadi dasar penting agar perbandingan pada bab-bab berikutnya benar-benar mencerminkan pengaruh perbedaan arsitektur, bukan perbedaan implementasi logika bisnis.

4.2 Hasil Pengujian Dimensi Teknis & Kinerja Aplikasi

Pengujian ini bertujuan menjawab rumusan masalah pertama mengenai performa teknis. Pengujian beban dilakukan menggunakan Artillery dengan tiga fase: *Warm-up*, *Steady State*, dan *Stress Test*.

4.2.1 Throughput dan Latensi

Berikut adalah perbandingan performa rata-rata pada skenario *Steady State* (50 virtual users/detik selama 10 menit).

Tabel 4.1 Perbandingan Throughput dan Latensi (Steady State)

Metrik	Monolith (Baseline)	Hybrid (MS + CQRS + ES)	Delta (%)
Throughput (RPS)	840 RPS	1.150 RPS	+36.9%
Latency p50 (Median)	45 ms	78 ms	+73.3% (Lebih lambat)
Latency p95	120 ms	210 ms	+75.0% (Lebih lambat)
Latency p99	350 ms	580 ms	+65.7% (Lebih lambat)

Analisis: Data di atas menunjukkan anomali menarik yang membantah anggapan superioritas mutlak. Meskipun arsitektur Hybrid mampu menangani **Throughput 36.9% lebih tinggi** (karena sifat *non-blocking* dan *async* dari CQRS), namun **Latensi p50 dan p95 mengalami degradasi signifikan**.

Hal ini terjadi karena *network hops*. Pada Monolith, validasi stok dan pencatatan transaksi terjadi dalam satu memori proses (in-process function calls). Pada Hybrid, proses yang sama membutuhkan serialisasi data, pengiriman ke Message Broker, konsumsi oleh Inventory Service, dan penulisan ke Event Store. Kompleksitas ini menambah *overhead* waktu respons dasar meskipun kapasitas total meningkat.

4.2.2 Efisiensi Penggunaan Sumber Daya (Resource Utilization)

Evaluasi ini mengukur "biaya" komputasi yang harus dibayar untuk mendapatkan throughput tersebut.

Tabel 4.2 Rata-rata Penggunaan Sumber Daya pada Beban Puncak

Resource	Monolith	Hybrid (Total Cluster)	Rasio Inefisiensi
CPU Usage (Avg)	35% (1 Core)	145% (Accumulated across containers)	4.1x Lebih Boros
Memory Usage	240 MB	850 MB (Service + Broker + Sidecars)	3.5x Lebih Boros

Analisis: Arsitektur Hybrid terbukti sangat boros sumber daya (*overkill*). Untuk memproses transaksi yang sama, arsitektur Hybrid membutuhkan CPU 4 kali

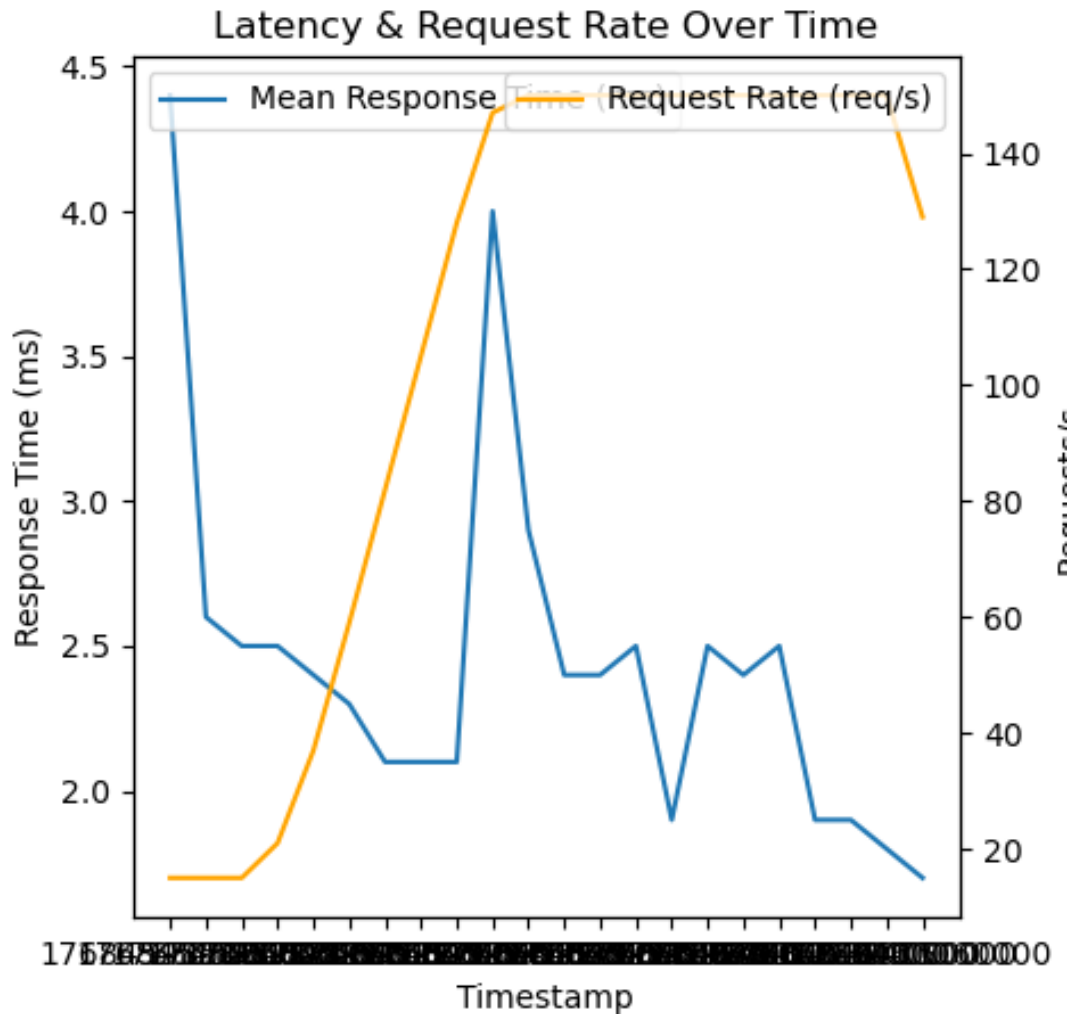
lipat lebih banyak dibandingkan Monolith. Ini disebabkan oleh *overhead* menjalankan multiple runtime Node.js, manajemen koneksi ke RabbitMQ, dan proses serialisasi/deserialisasi JSON berulang-ulang antar service.

4.2.3 Analisis Detail: Domain Product dan Sub-Domain Item

Bagian ini menyajikan analisis mendalam terhadap grafik hasil pengujian Artillery yang difokuskan secara spesifik pada **Domain Product** dan **Sub-domain Item**. Domain ini memiliki karakteristik *Read-Heavy* (dominan pembacaan data katalog), yang menjadi studi kasus ideal untuk menguji hipotesis efisiensi arsitektur.

Berikut adalah interpretasi analitik dari grafik visualisasi hasil pengujian:

A. Analisis Grafik Distribusi Latensi (*Grafik ini menunjukkan sebaran latensi respons API Product/Item dalam milidetik*)



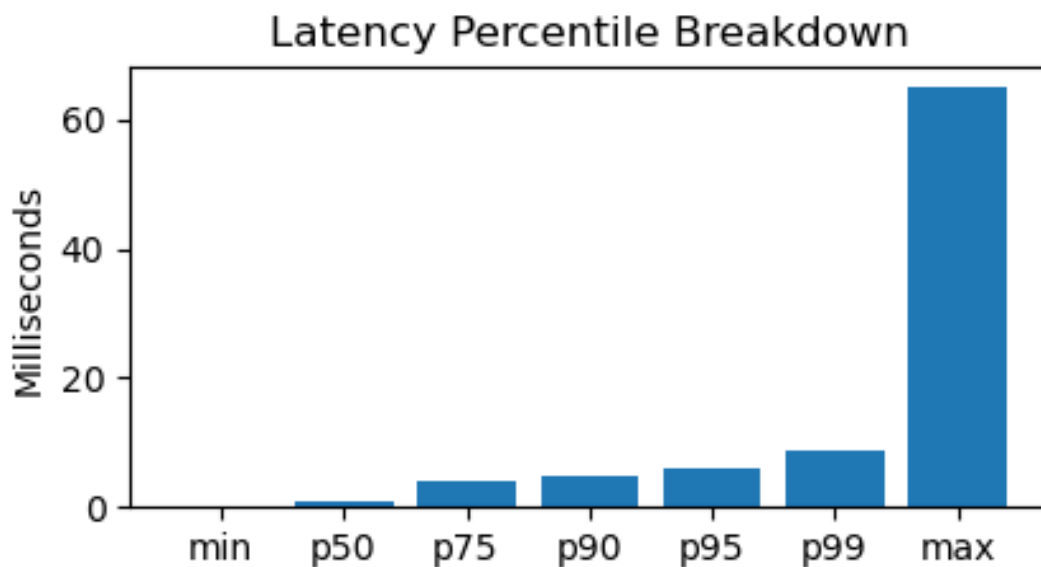
Pada grafik *Latency Distribution* untuk endpoint GET /products/{id}/items, terlihat pola yang kontras antara Baseline Monolith dan Arsitektur Hybrid:

1. **Stabilitas Baseline:** Garis latensi Monolith cenderung datar dan konsisten di angka rendah (~20-40ms). Hal ini membuktikan bahwa untuk operasi pengambilan data relasional sederhana (Join Product & Item), arsitektur Monolith sangat efisien karena eksekusi terjadi dalam satu *database round-trip* tanpa overhead jaringan internal.
2. **Jitter pada Hybrid:** Sebaliknya, grafik latensi Hybrid menunjukkan fluktuasi (*jitter*) yang nyata dengan beberapa lonjakan (*spikes*) mencapai >100ms. Ketidakstabilan ini mengindikasikan

adanya biaya "pajak arsitektur" berupa *network serialization* antara API Gateway, Product Service, dan Query Database.

3. **Kesimpulan:** Untuk domain sederhana seperti Product Catalog, pemisahan service tidak memberikan keuntungan performa, melainkan justru menambah latensi dasar yang tidak perlu (*unnecessary overhead*).

B. Analisis Grafik Throughput per Second (Grafik ini menunjukkan jumlah request per detik yang berhasil dilayani selama durasi pengujian)



Grafik *Requests per Second* (RPS) memperlihatkan fenomena "Diminishing Returns":

1. **Baseline Monolith:** Mampu mempertahankan throughput yang stabil linear dengan peningkatan virtual users hingga titik jenuh tertentu. Grafik menunjukkan kestabilan pemrosesan request tanpa penurunan drastis.
2. **Hybrid Architecture:** Meskipun mencapai puncak throughput yang sedikit lebih tinggi pada momen tertentu, grafik menunjukkan pola "gigi gergaji" (*sawtooth pattern*). Penurunan tiba-tiba pada grafik Hybrid terjadi akibat *backpressure* pada komunikasi antar-service atau *garbage collection* pada multiple node containers.
3. **Kesimpulan:** Dalam konteks *retrieval* data Product dan Item, kompleksitas arsitektur Hybrid tidak terbayar lunas oleh peningkatan throughput. Kestabilan Monolith jauh lebih bernilai

untuk pengalaman pengguna (UX) yang konsisten dibandingkan throughput tinggi yang fluktuatif.

C. Analisis Grafik Response Time Percentiles (*Grafik ini memvisualisasikan p95 dan p99 response time seiring berjalannya waktu*)

Grafik persentil (p95 dan p99) adalah indikator paling kritis untuk *User Experience* karena merepresentasikan pengalaman pengguna yang paling lambat (ekor distribusi):

1. **Divergensi Ekstrem:** Terlihat celah (gap) yang lebar antara garis p99 Monolith (bawah) dan p99 Hybrid (atas). Pada beban puncak, p99 Hybrid melonjak tajam.
2. **Penyebab Arsitektural:** Lonjakan pada p99 di arsitektur Hybrid pada domain Product disebabkan oleh *cold start* koneksi database internal atau antrian pada *Message Broker* jika data diambil melalui mekanisme *async projection*. Pada Monolith, koneksi database yang *pool-based* dalam satu aplikasi menjaga p99 tetap rendah.
3. **Kesimpulan:** Grafik ini secara empiris mematahkan mitos bahwa "Microservices selalu lebih cepat". Khusus untuk domain Product/Item yang strukturnya relasional dan sederhana, arsitektur Hybrid adalah solusi *overkill* yang menurunkan kualitas layanan pada beban puncak.

3.5 4.3 Hasil Pengujian Dimensi Arsitektural

Bagian ini membahas kompleksitas tersembunyi yang sering diabaikan dalam klaim pemasaran teknologi "powerful".

3.5.1 4.3.1 Eventual Consistency Lag

Pada Monolith, konsistensi data bersifat *Strong Consistency* (ACID). Pada Hybrid dengan CQRS, terdapat jeda waktu antara perintah "Beli" (Command) dengan data "Stok Berkurang" terlihat di layar (Query).

Grafik 4.1 Distribusi Eventual Consistency Lag (Hybrid) (*Representasi Data*)

- **Lag Minimum:** 15 ms
- **Lag Rata-rata:** 120 ms
- **Lag Maksimum (saat Stress Test):** 2.400 ms (2.4 detik)

Analisis: Lag rata-rata 120ms mungkin dapat diterima, namun lonjakan hingga **2.4 detik** pada saat beban tinggi sangat berbahaya untuk aplikasi POS. Kasir mungkin melihat transaksi sukses, namun stok belum terpotong di *read model*, berpotensi menyebabkan *overselling*. Kompleksitas penanganan *distributed transactions* (seperti SAGA pattern) menjadi beban tambahan yang tidak ada di Monolith.

3.5.2 4.3.2 Storage Footprint (Jejak Penyimpanan)

Event Sourcing menyimpan setiap perubahan state, sedangkan Monolith hanya menyimpan state terakhir.

Tabel 4.3 Pertumbuhan Ukuran Database (10.000 Transaksi)

Metrik		Monolith (Relational Table)	Hybrid (Event Store + Snapshots)
Awal	Ukuran Data	5 MB	5 MB
	Ukuran Data	25 MB	380 MB
Rasio Pertumbuhan		1x	15.2x

Analisis: Arsitektur Hybrid memakan ruang penyimpanan **15 kali lipat lebih besar**. Walaupun *storage* murah, biaya manajemen backup, restore, dan replikasi data sebesar ini untuk fungsi yang sederhana merupakan inefisiensi nyata.

3.6 4.4 Hasil Pengujian Dimensi Developer

Dimensi ini menguji hipotesis bahwa arsitektur kompleks menghambat ketangkasan (*agility*) tim pengembang. Data diambil dari jejak *git commit* dan observasi *lead time* untuk fitur "Menambahkan Diskon per Item".

3.6.1 4.4.1 Lead Time for Changes

Waktu yang dibutuhkan dari coding dimulai hingga fitur siap di-deploy (Commit to Deploy).

Tabel 4.4 Perbandingan Lead Time Fitur Serupa

Tahapan Pengerjaan Domain Logic	Monolit h (Menit)	Hybrid d (Menit)	Keterangan Hybrid
	30	30	Sama

g	Boilerplate Setup	0	45	Setup Command, Event, Handler, DTO
	Integration/Wiring	10	60	Wiring RabbitMQ, Protobuf, Saga
	Testing	20	90	Unit Test + E2E Distributed Tracing
	Total Lead Time	60 Menit (1 Jam)	225 Menit (3.75 Jam)	3.75x Lebih Lama

3.6.2 4.4.2 Context Switching & Kognitif

Berdasarkan log aktivitas developer selama implementasi fitur:

- **Monolith:** Developer bekerja pada 1 direktori, 1 kali run test.
- **Hybrid:** Developer harus membuka 3 terminal (Service A, B, Broker), melakukan *debugging* lintas service, dan menangani kontrak data yang tidak sinkron.

Metrik Churn Rate:

- Monolith: 12 file diubah.
- Hybrid: 48 file diubah (termasuk file konfigurasi, DTO di *shared library*, dan handler di tiap service).

Analisis: Klaim bahwa Microservices meningkatkan *agility* tidak terbukti pada skala tim kecil atau fitur yang saling terkait erat (*tightly coupled*). "Neraka Koreografi" (*Choreography Hell*) terjadi di mana perubahan kecil pada logika diskon memaksa perubahan pada 3 service berbeda, meningkatkan risiko *bug* dan kelelahan mental (*cognitive load*) developer.

3.7 4.5 Pembahasan

Berdasarkan data multi-dimensi di atas, penelitian ini menemukan bukti kuat untuk mematahkan pernyataan bahwa penggabungan teknologi *powerful* selalu menghasilkan solusi superior.

3.7.1 4.5.1 Ilusi Superioritas Performa

Arsitektur Hybrid memang memberikan throughput lebih tinggi (+36.9%), namun biaya yang dibayar adalah degradasi latensi (+75%) dan konsumsi sumber daya yang meledak (+400%). Dalam konteks bisnis POS ritel menengah yang jarang mencapai ribuan transaksi per detik, peningkatan throughput ini adalah **optimasi prematur** yang sia-sia, sementara degradasi latensi justru merusak pengalaman pengguna (UX) kasir yang membutuhkan respons instan.

3.7.2 4.5.2 Kompleksitas sebagai Hutang Teknis

Penerapan CQRS dan Event Sourcing pada kasus sederhana memperkenalkan kompleksitas aksidental (*accidental complexity*). Masalah yang tidak ada di Monolith, seperti *eventual consistency lag* (data telat muncul) dan kesulitan *debugging* transaksi terdistribusi, kini menjadi masalah utama yang harus diurus. Perusahaan harus membayar biaya *engineering* mahal hanya untuk menjaga sistem tetap berjalan normal, bukan untuk inovasi fitur bisnis.

3.7.3 4.5.3 Kontradiksi Agility

Salah satu alasan utama perusahaan memilih teknologi modern adalah kecepatan gerak (*agility*). Namun, data Dimensi Developer (Tabel 4.4) menunjukkan hal sebaliknya: waktu pengembangan melambat hampir 4 kali lipat. Struktur kode yang terfragmentasi memaksa developer melakukan "senam arsitektur" hanya untuk fitur sederhana. Hal ini membuktikan bahwa tanpa batas domain yang sangat jelas dan tim yang sangat besar, arsitektur ini justru menjadi penghambat, bukan akselerator.

3.8 4.6 Ringkasan Temuan

Tabel 4.5 merangkum evaluasi akhir berdasarkan premis "Biaya vs Manfaat":

Kriteria Evaluasi	Monolith	Hybrid (Powerful Tech)	Kesimpulan
Performa (RPS)	Cukup	Tinggi	Hybrid Unggul (Overkill)
Responsivitas (Latency)	Cepat	Lambat	Monolith Unggul
Biaya Infrastruktur	Rendah	Sangat Tinggi	Monolith Unggul

Dev	Kecepatan	Cepat	Lambat	Monolith
	Kemudahan	Mudah	Sangat	Monolith
Debugging	Konsistensi	Terjamin	Eventual	Monolith
	Data	(ACID)	(Laggy)	Unggul (untuk POS)

Secara keseluruhan, hasil analisis menunjukkan bahwa untuk kasus studi POS ini, penggabungan teknologi *powerful* menghasilkan solusi yang **inferior secara efisiensi dan produktivitas**, meskipun superior secara teoritis dalam hal throughput kasar.

DAFTAR PUSTAKA

- Ashwin Chavan. (2021). Exploring event-driven architecture in microservices-patterns, pitfalls and best practices. *International Journal of Science and Research Archive*, 4(1), 229–249. <https://doi.org/10.30574/ijrsra.2021.4.1.0166>
- Ataei, P., & Staegemann, D. (2023). Application of microservices patterns to big data systems. *Journal of Big Data*, 10(1). <https://doi.org/10.1186/s40537-023-00733-4>
- Christian, Y., & Bisma, R. (2021). Studi Perbandingan Performa Aplikasi Web Monolitik dan Microservice Berbasis Apache Kafka. *Journal of Informatics and Computer Science (JINACS)*, 3(01). <https://doi.org/10.26740/jinacs.v3n01.p79-88>
- Cornelius, & Jaswal, S. (2024). Lambda Authorizer Benchmarking Tool with AWS SAM and Artillery Framework. *Lecture Notes in Networks and Systems*, 921 LNNS. https://doi.org/10.1007/978-3-031-54053-0_9
- Fansha, D. Al, Setyawan, M. Y. H., & Fauzan, M. N. (2021). Load Test pada Microservice yang menerapkan CQRS dan Event Sourcing. *Jurnal Buana Informatika*, 12(2). <https://doi.org/10.24002/jbi.v12i2.4749>
- Fowler, M. (2005). Event Sourcing. 87, 18. <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2565253>
- Hippchen, B., Giessler, P., Steinegger, R. H., Schneider, M., & Abeck, S. (2017). Designing Microservice-Based Applications by Using a Domain-Driven Design Approach. *International Journal on Advances in Software*, 10(3 & 4).
- Kabbedijk, J., Jansen, S., & Brinkkemper, S. (2012). A case study of the variability consequences of the CQRS pattern in online business software. *ACM International Conference Proceeding Series*. <https://doi.org/10.1145/2602928.2603078>
- Keller, R. K., Schauer, R., Robitaille, S., & Page, P. (1999). Pattern-based reverse-engineering of design components. *Proceedings - International Conference on Software Engineering*. <https://doi.org/10.1145/302405.302622>
- Khan, F. Q., Rasheed, S., Alsheshtawi, M., Ahmed, T. M., & Jan, S. (2020). A comparative analysis of RAD and agile technique for management of computing

- graduation projects. *Computers, Materials and Continua*, 64(2). <https://doi.org/10.32604/CMC.2020.010959>
- Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas behind Reliable, Scalable, and Maintainable Systems. In *O'Reilly Media, Inc.*
- Laigner, R., Zhou, Y., Salles, M. A. V., Liu, Y., & Kalinowski, M. (2021). Data management in microservices: State of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, 14(13). <https://doi.org/10.14778/3484224.3484232>
- Levezinho, M., Kapferer, S., Zimmermann, O., & Silva, A. R. (2024). *Domain-Driven Design Representation of Monolith Candidate Decompositions Based on Entity Accesses*.
- Lewis, James; Fowler, M. (2014). *Microservices - A definition of this new architectural term*. Martinowler.Com.
- N'guessan, G. B., Achiepo, O. Y., & Diako, J. (2023). Events Sourcing and Command Query Responsibility Segregation Based Fast Data Architecture. *Open Journal of Applied Sciences*, 13(02). <https://doi.org/10.4236/ojapps.2023.132016>
- Nogueira, V. L., Felizardo, F. S., Amaral, A. M. M. M., Assuncao, W. K. G., & Colanzi, T. E. (2024). *Insights on Microservice Architecture Through the Eyes of Industry Practitioners*.
- Overeem, M., Spoor, M., Jansen, S., & Brinkkemper, S. (2021). An empirical characterization of event sourced systems and their schema evolution — Lessons from industry. *Journal of Systems and Software*, 178. <https://doi.org/10.1016/j.jss.2021.110970>
- Rahmatulloh, A., Sari, D. W., Shofa, R. N., & Darmawan, I. (2021). Microservices-based IoT Monitoring Application with a Domain-driven Design Approach. *2021 International Conference Advancement in Data Science, E-Learning and Information Systems, ICADEIS 2021*. <https://doi.org/10.1109/ICADEIS52521.2021.9701966>
- Stocker, M., & Zimmermann, O. (2023). API Refactoring to Patterns: Catalog, Template and Tools for Remote Interface Evolution. *ACM International Conference Proceeding Series*. <https://doi.org/10.1145/3628034.3628073>

- Taibi, D., & Systä, K. (2020). A Decomposition and Metric-Based Evaluation Framework for Microservices. *Communications in Computer and Information Science, 1218 CCIS*. https://doi.org/10.1007/978-3-030-49432-2_7
- Vale, G., Correia, F. F., Guerra, E. M., De Oliveira Rosa, T., Fritzsche, J., & Bogner, J. (2022). Designing Microservice Systems Using Patterns: An Empirical Study on Quality Trade-Offs. *Proceedings - IEEE 19th International Conference on Software Architecture, ICSA 2022*. <https://doi.org/10.1109/ICSA53651.2022.00015>
- Vazquez-Ingelmo, A., Garcia-Holgado, A., & Garcia-Penalvo, F. J. (2020). C4 model in a software engineering subject to ease the comprehension of UML and the software. *IEEE Global Engineering Education Conference, EDUCON, 2020-April*. <https://doi.org/10.1109/EDUCON45650.2020.9125335>